



Machine Learning Foundations for Risk and Finance

Autumn 2026

Hoang Viet Le
lehoangviet03@gmail.com

Keynum
INVESTMENTS

About myself



- **Education:**
 - PhD candidate, Paris Saclay (current)
 - Master in Fintech, Paris 1 Pantheon Sorbonne
 - Master in Quantitative Finance and Risk Management, CYTech
- **Professional Career:** Lead Quantitative Researcher and Data Scientist, Keynum Investments
- **Expertise:**
 - Decision Support Systems
 - Quantitative Trading Systems

Course Objective



- This course provides a comprehensive introduction to foundational ML concepts
- We will explore the complete machine learning workflow, from data pre-processing to model evaluation
- **Objective:** Equip students with the practical skills to select, build, and evaluate appropriate models
- While broadly applicable, special attention will be given to use-cases within the financial sector

Course Requirements

- Laptop and Internet Connection are required for coding (Using google colab)
- Attendance and homework will be part of the final grade.

Syllabus

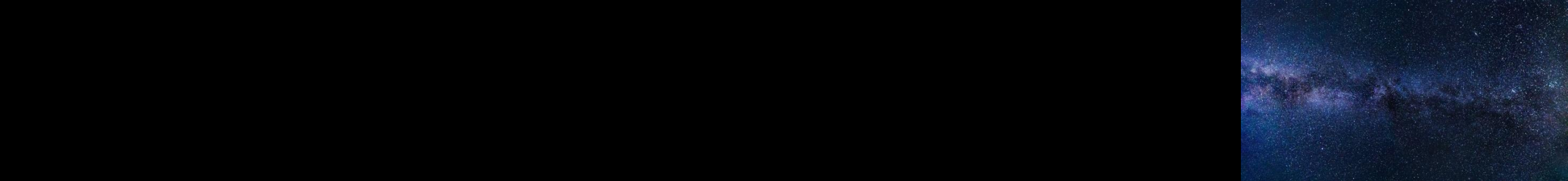


- Week 1: Intro & Linear Regression Models
 - ML in Finance
 - Supervised Workflow & Data Leakage
 - Linear Regression, Ridge, Lasso, and ElasticNet
- Week 2: Classification & Advanced Evaluation
 - Logistic Regression
 - K-Nearest Neighbors; Thresholds, Costs & Calibration
- Week 3: Advanced Classifiers & Clustering
 - Support Vector Machines (SVMs)
 - K-Means and DBSCAN Clustering

Syllabus



- Week 4: Tree-Based & Ensemble Methods
 - Decision Trees
 - Bagging (Random Forest)
 - Boosting (AdaBoost, Gradient Boosting)
 - Bias-Variance Tradeoff
- Week 5: Dimensionality Reduction & XAI
 - Principal Component Analysis (PCA)
 - Explainable AI (XAI); Foundation Models (outlook)
- Week 6: Final Project Presentations



Machine Learning Applications in Fintech

Artificial Intelligence & Machine Learning

What is the difference between Artificial Intelligence (AI) and Machine Learning?

- The two terms are often used interchangeably but they are not the same
- AI is a broader concept to create intelligent machines that can simulate human thinking capability and behaviour. Its action and behaviour can be pre-programmed and based on rules but not necessary.
- Machine learning enables a computer system to make predictions or take some decisions using historical data without being explicitly programmed.
- Expert, rule-based systems are part of AI but not machine learning.

Machine Learning



- The finance industry is one of the pioneers to incorporate AI technologies for:
 - Algorithmic trading and portfolio optimization
 - Credit scoring
 - Fraud detection, KYC
 - Risk management
- There are three main learning paradigms in machine learning:
 - **Supervised learning**
 - **Unsupervised learning**
 - **Reinforcement learning**

Algorithmic Trading & Portfolio Management

- Reinforcement learning for optimizing trading strategies
- Natural Language Processing (NLP) for sentiment analysis of financial news
- Example: JPMorgan's LOXM AI system for equities trading

Enhanced Credit Scoring

- Alternative data sources for credit assessment (social media, utility bills)
- Machine learning models for more accurate risk assessment
- Example: Upstart's AI-powered lending platform

Advanced Fraud Detection



- Real-time transaction monitoring using deep learning models
- Anomaly detection with unsupervised learning algorithms
- Example: Feedzai's AI platform for fraud prevention

Personalized Banking Experience

- AI-driven chatbots and virtual assistants (e.g., Bank of America's Erica)
- Tailored product recommendations using collaborative filtering
- Predictive analytics for customer churn prevention

Regulatory Compliance (RegTech)

- Automated regulatory reporting using NLP and machine learning
- AI-powered KYC (Know Your Customer) and AML (Anti-Money Laundering) processes
- Example: ComplyAdvantage's AI-driven compliance solutions

Insurance (InsurTech)

- AI for automated claims processing and fraud detection
- Machine learning for personalized policy pricing
- Example: Lemonade's AI Jim for ultra-fast claims handling

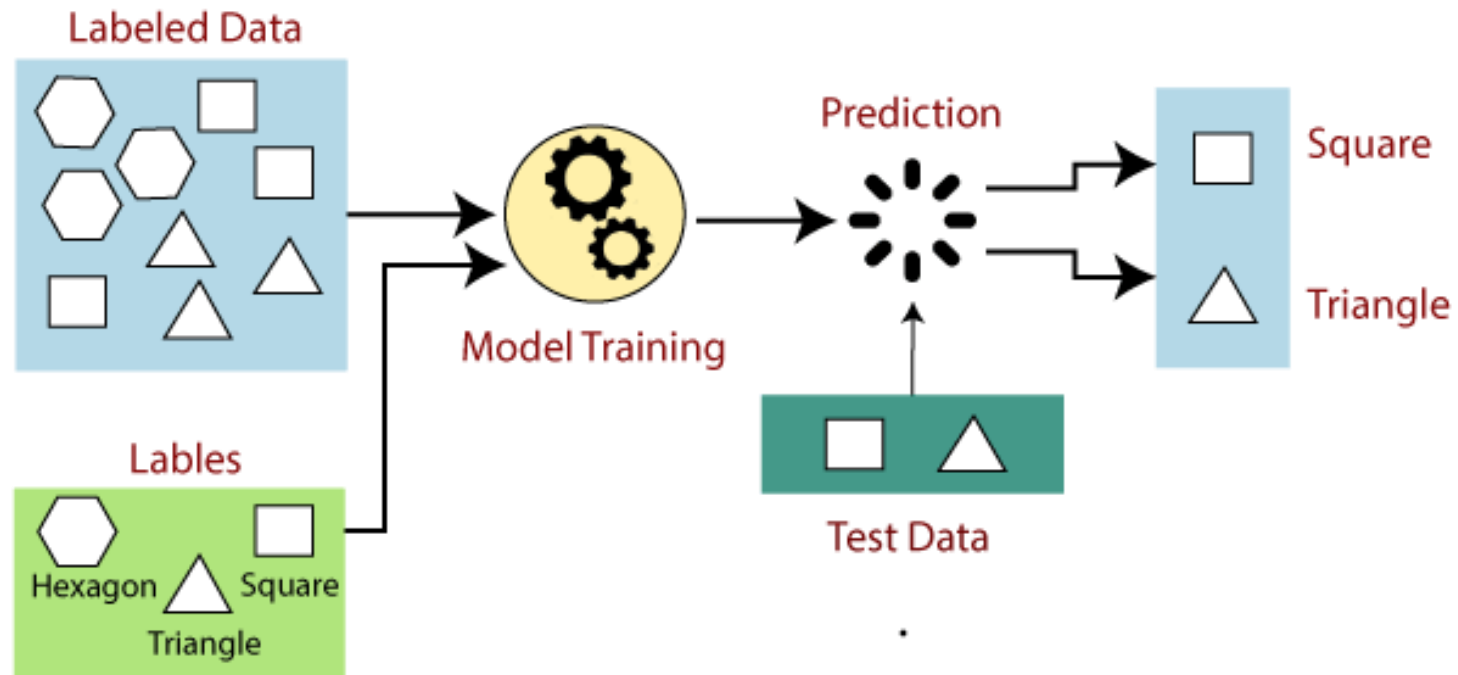
Supervised Learning



- Training data is pre-labelled or partially pre-labelled (semi-supervised)
- The objective is to find an algorithm (function) to map between the inputs x (features) and the outputs y (labelled data) and to make predictions about new, unseen data.
- The output y can either be continuous or discrete
 - Regression problem: continuous output
 - Classification problem: discrete output
 - Binary classification: output is True or False (for example, if an image is a car or not)
 - Multiclass classification: outputs are different classes (for example, which kind of flowers)
- For each type of problem, there are different metrics that can be used:
 - R^2 , MSE, MAE, etc. for regression
 - Accuracy, Log loss, Precision, Recall, F1 score, etc. for classification

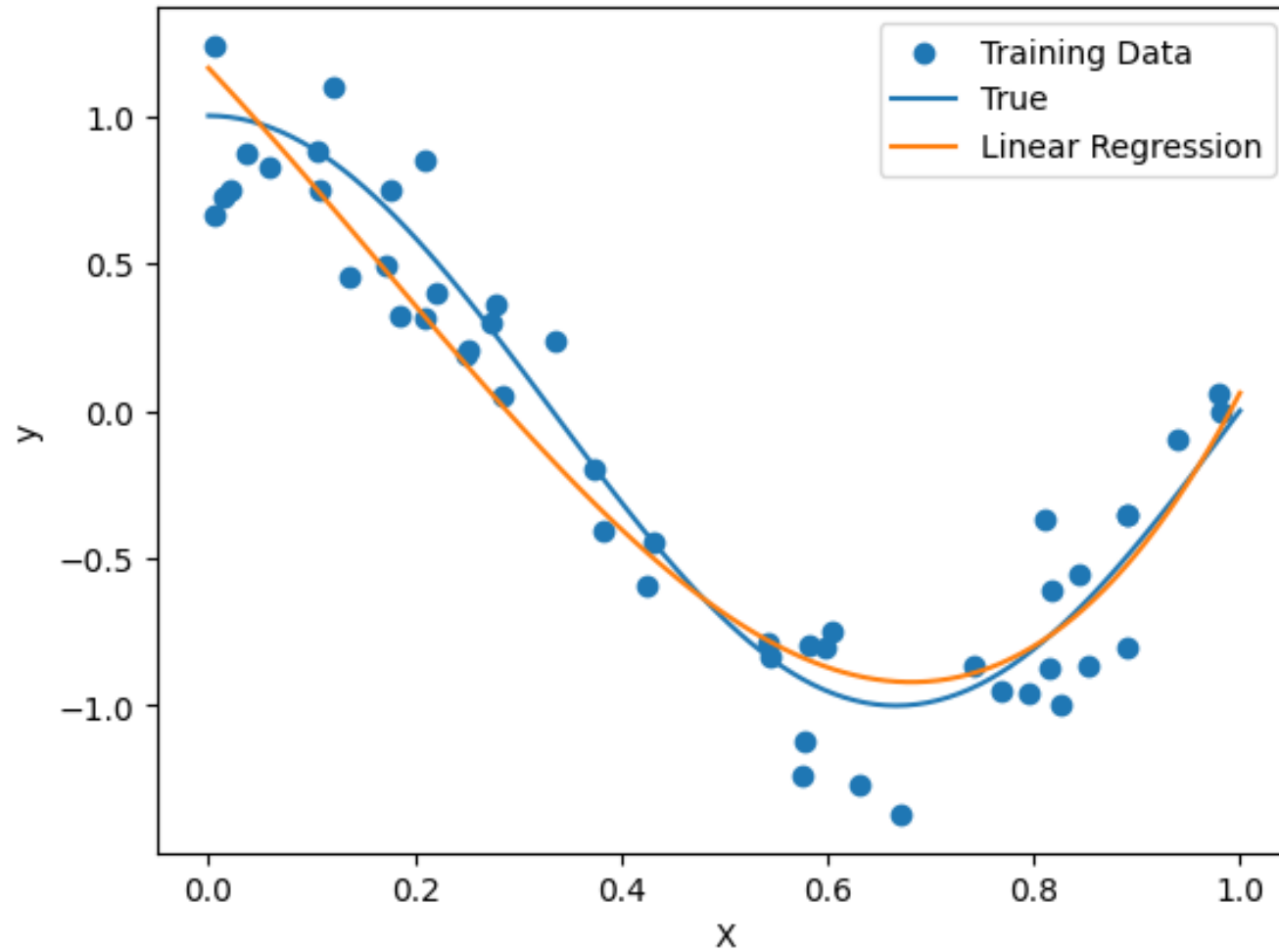
Supervised Learning

Example of Supervised learning



Source: javatpoint.com

Supervised Learning

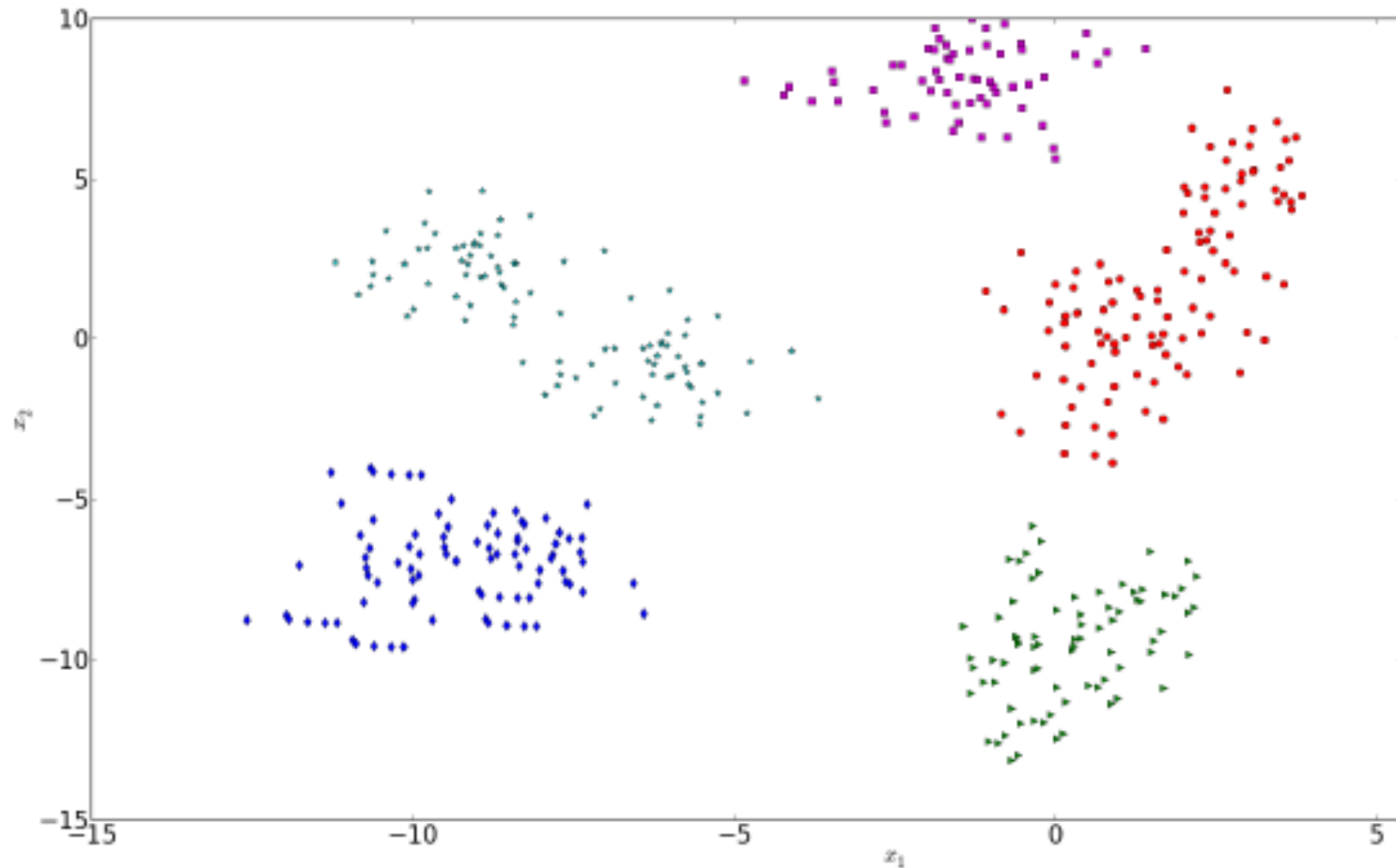


Unsupervised Learning



- Training data is not labelled
- The objective is to find patterns or relationship in the data without any prior knowledge of the outputs.
- Two types of unsupervised learning:
 - Clustering: dividing the data into several groups with similar characteristics based on their features
 - Dimensionality reduction: reducing the number of features of data while retaining as much information as possible (PCA technique)

Unsupervised Learning



Example of Clustering

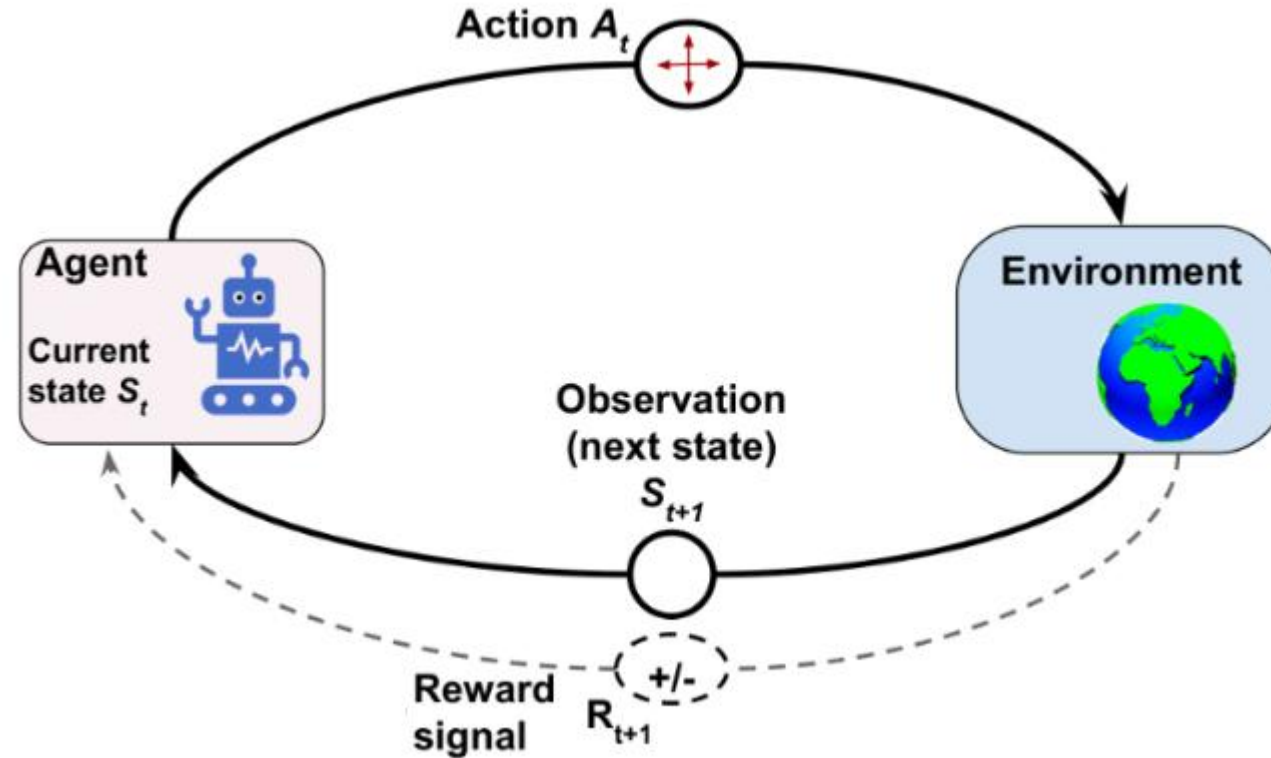
Reinforcement Learning



- Training data is not labelled
- The objective is to find the optimal output through a trial and error process.
- The output is actions which are evaluated by a predefined agent when they interact with the environment the model is placed in.
- The evaluation of these actions, which can be a reward or a punishment, works as a feedback for the model and helps the model to improve.
- Example:
 - AlphaGo
 - ChatGPT is the combination of both supervised and reinforcement learning

Reinforcement Learning

Example of Reinforcement learning



Source: vitalflux.com

Steps in Supervised Learning



- Splitting training and test dataset: do this first. Decide the evaluation strategy, then hold the test set aside until the very end.
- Pre-processing data: normalization and transformation, fitted on the training data only.
- Feature Engineering: Create new features from existing ones and select the most relevant features based on domain knowledge or statistical methods
- Model Selection and Training: where the choice of supervised learning algorithm becomes relevant. "Training" means your data set is used in optimization of the loss function to find parameters for $f(x)$
- Hyperparameter Tuning: Finding the best set of parameters for your models
- Model Evaluation: This is about which metrics to use to evaluate performance and how to calculate those metrics.
- Model Interpretation and Visualization
- Model Deployment and Monitoring

Data Pre-processing



- Data exploration to see how variables and samples relate to each other. We might want to check the correlation between predictor variables and keep only one variable from that group.
- Dealing with missing values
- Removing outlier
- Feature scaling and normalization
- Data transformation (if needed for specific algorithms)

Dealing with missing values



- Removing the sample with missing values
- Replacing the missing values with the mean or median
- Replacing the missing values with the nearest neighbours or k-nearest neighbours
- Predicting the missing values using a separate machine learning models

Data Transformation



- Categorical features: nominal or ordinal => converted to dummy variable or not
- Cyclical features: considering sine, cosine transformation or embedding
- Beware of non-stationarity in time series
- Normalization (Min-Max Scaling): linear rescaling to a fixed range, output from 0 to 1

$$X_{Scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- Standardization: centre and rescale, output with mean 0, std 1 (no Gaussian assumption)

$$X_{Scaled} = \frac{X - \mu}{\sigma}$$

Feature Engineering



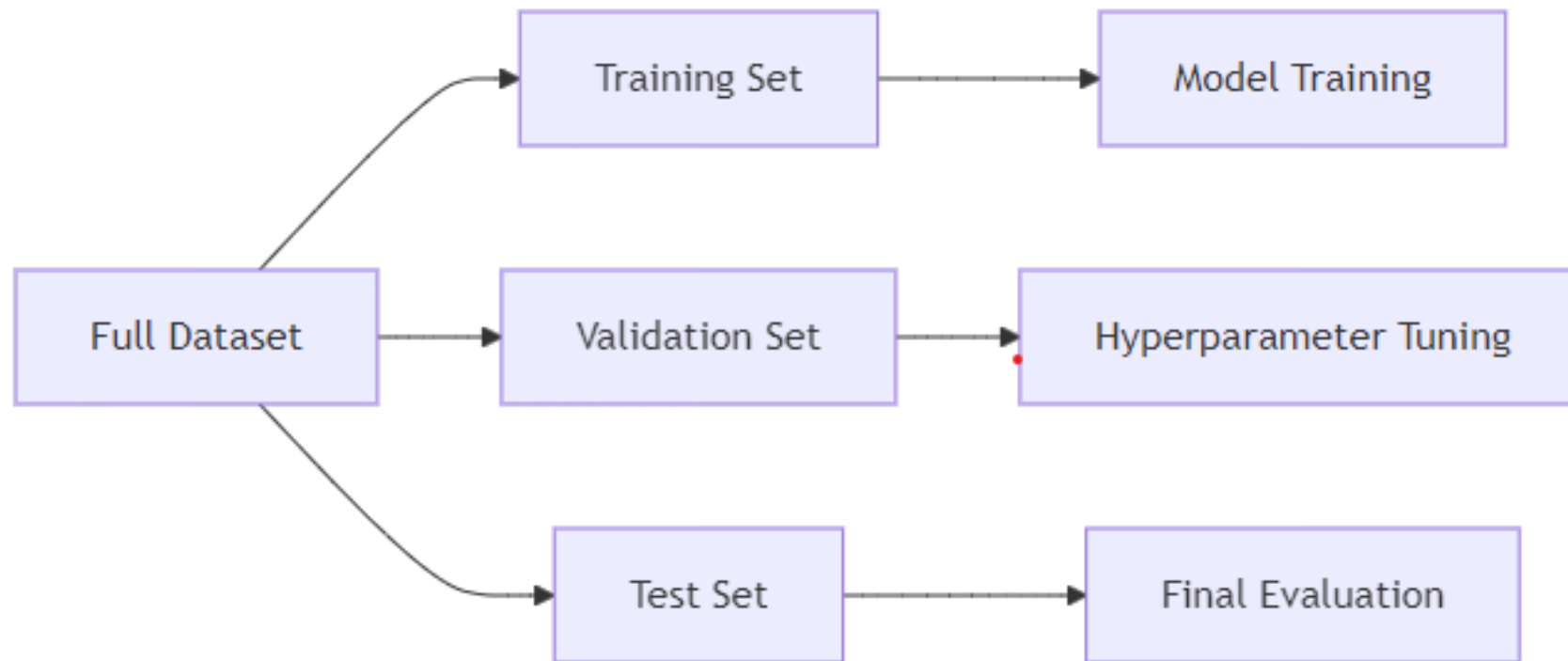
- Create new features from existing ones
- For time series data: create lagged features, moving averages, etc.
- Encode categorical variables
- Select relevant features based on domain knowledge or statistical methods

Data Splitting

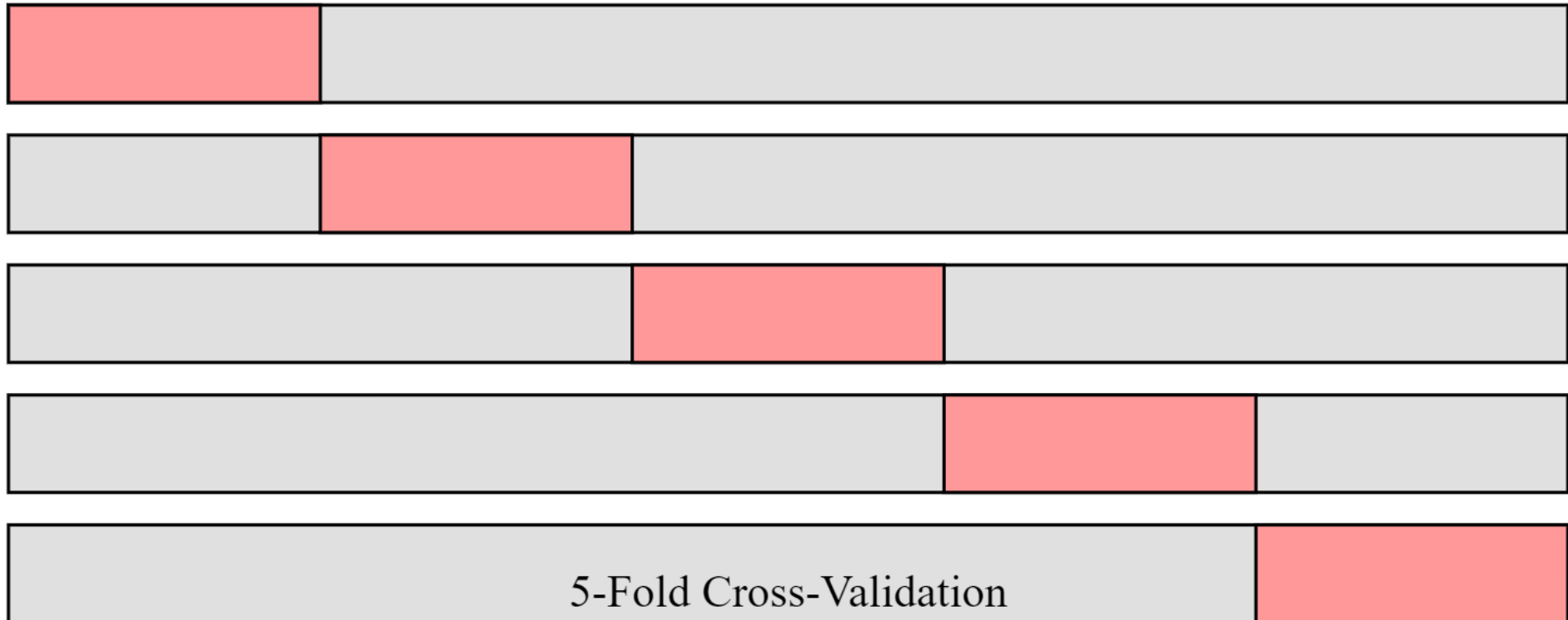


- Without a separate test set, we cannot assess the performance of our model or tune it properly
- Sometimes, a validation set is required for hyperparameter tuning (Train-Validation-Test)
- Approaches:
 - Splitting into separated set (using a specific ratio, i.e. 70:30 or 50:25:25)
 - Cross-validation:
 - Splitting the data into randomly sampled k subsets, called k -folds
 - Build models and estimate errors k times, each time uses $k-1$ groups to predict the remaining group.
 - Bootstrap resampling: draw with replacement; the unselected points form the test set.
- Pay attention to data shuffling or stratified sampling. Sometimes, data should not be randomized.
- For time series use `TimeSeriesSplit`; for panel data, split on a date shared by all entities.

Data Splitting



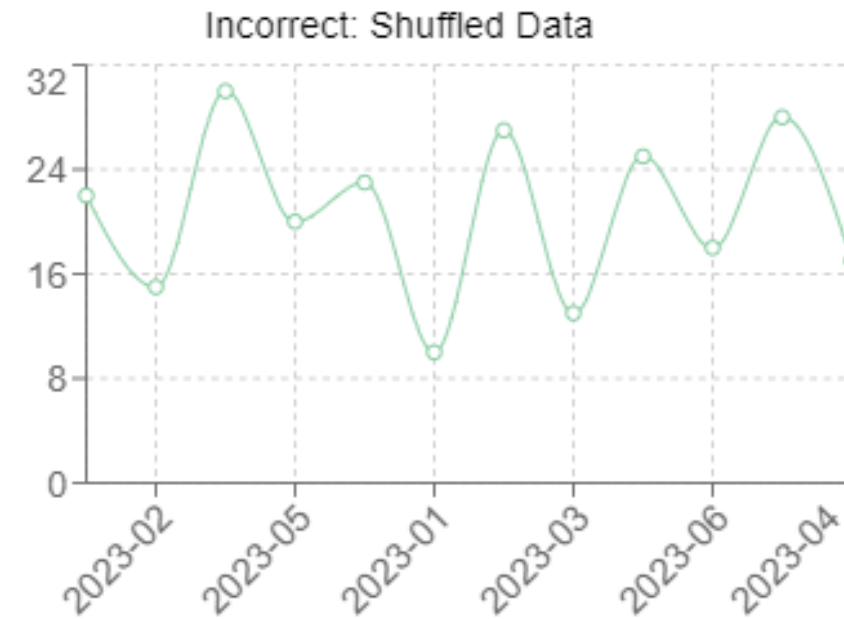
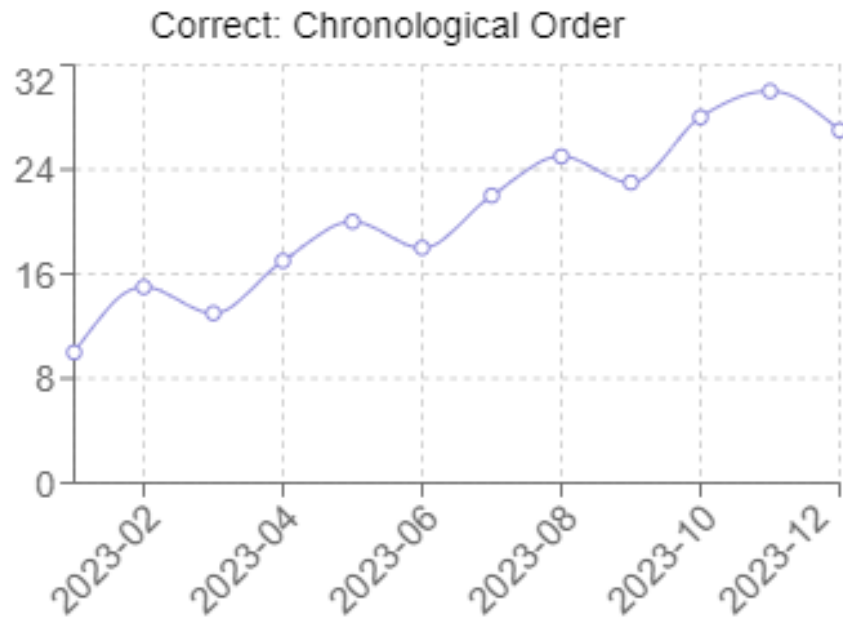
Data Splitting



Data Splitting



Time Series Data: Ordered vs Shuffled



Split First, Then Fit: Avoiding Data Leakage

- Leakage: information from the test set reaches the training process. The score flatters you.
- Any step that estimates something from the data is part of the model:
 - imputation values, scaling mean and std, PCA components, feature selection, encodings
- Wrong: `fit_transform` on the whole dataset, then split.
- Right: fit on the training fold, then transform validation and test with those parameters.
- The test set is opened once, at the end, to report the final number.
- In finance this is look-ahead bias: a perfect backtest that cannot be reproduced live.

Pipelines: Making the Correct Order Automatic

- A Pipeline chains the preprocessing steps and the model into a single estimator.
- `fit()` applies every step to the training data only; `predict()` reuses what was learned.
- Inside `GridSearchCV` the whole pipeline is refitted on each fold, so no fold can leak.

```
pipe = Pipeline([("scale", StandardScaler()),  
                 ("pca", PCA(n_components=10)),  
                 ("model", Ridge())])
```

```
grid = GridSearchCV(pipe, param_grid, cv=cv).fit(X_train, y_train)
```

- If you call `.fit()` on data that contains validation or test rows, something is wrong.

Cross-Validation for Time Series and Panel Data

- k-fold shuffles the rows, so a fold can train on the future and validate on the past.
- TimeSeriesSplit: each fold trains on the past and validates on the block after it.

```
cv = TimeSeriesSplit(n_splits=5)
```

- Panel data (the 15 plants of the project): split on a common date, not on rows.
 - every plant then shares the same train / validation boundary
- Build lag and rolling features inside each entity, never seeing the current value:

```
df.groupby("Plant_ID")["y"].shift(1).rolling(24).mean()
```
- shift(1) comes before rolling(...). Rolling then shifting still leaks the current value.

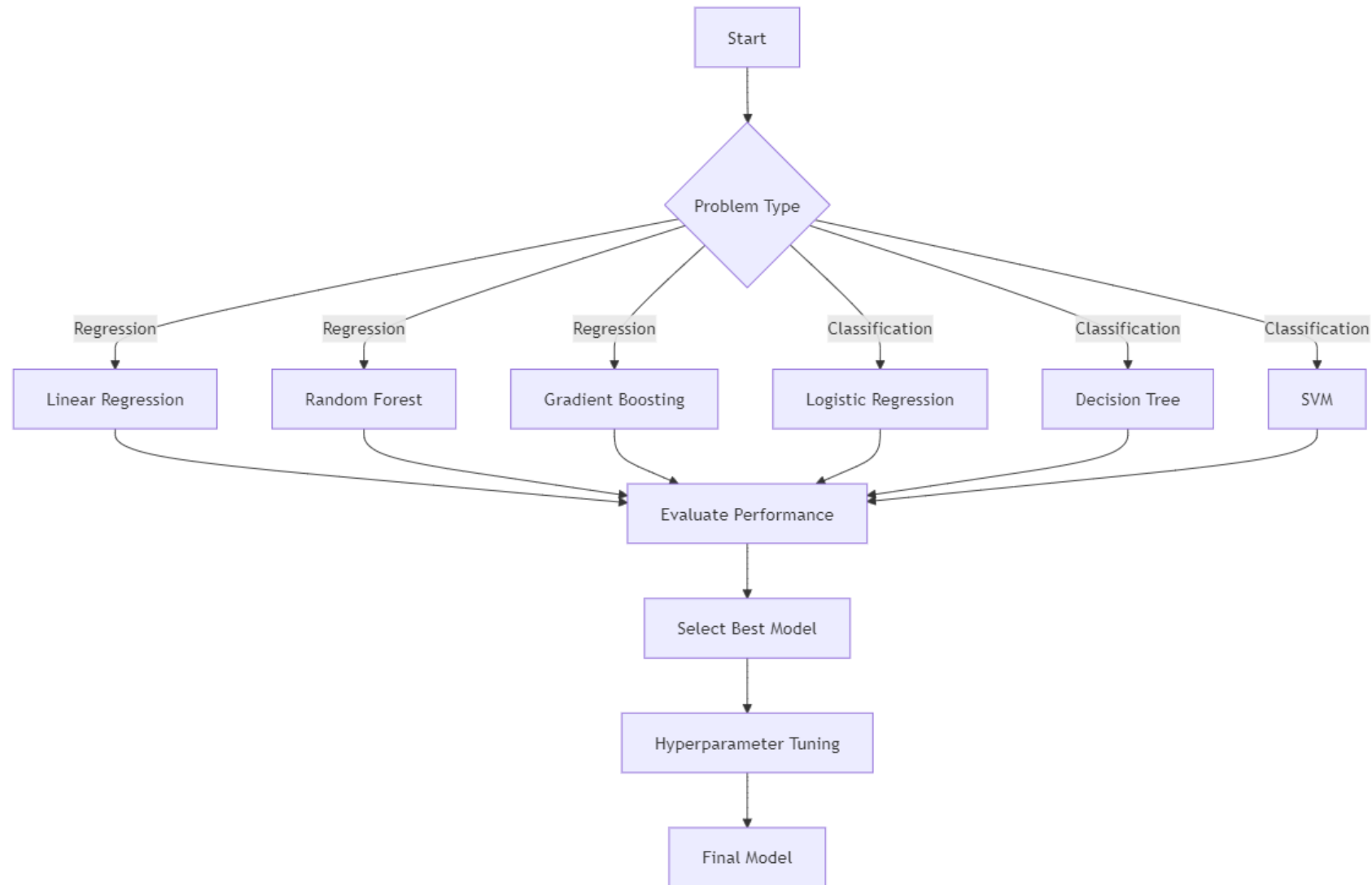
Model Selection and Training



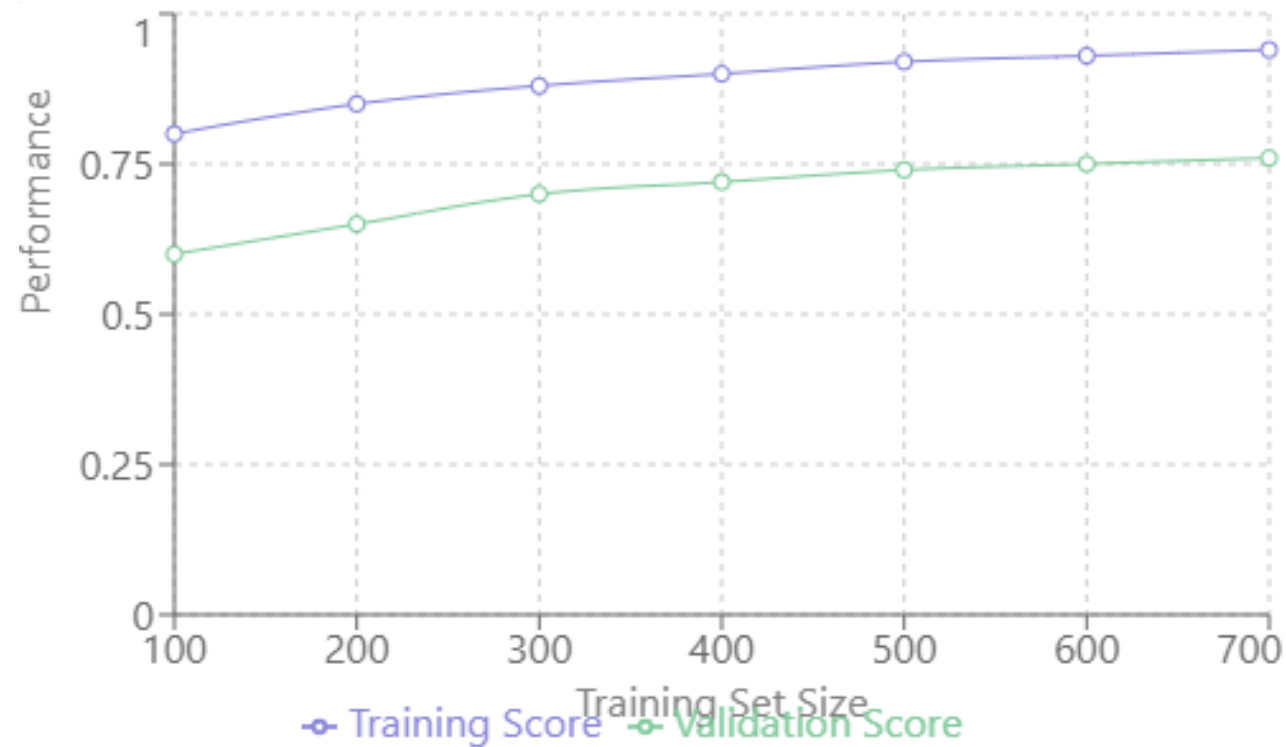
Choose appropriate algorithms based on the problem (classification or regression)

- Linear models: Linear Regression, Logistic Regression, Ridge, Lasso, ElasticNet, etc.
- K-Nearest Neighbours (K-NN)
- Support Vector Machine (SVM)
- Tree-based models: Decision Tree, Random Forest, Gradient Boosting, XGBoost, LightGBM, etc.
- Neural Networks: MLP, CNN, RNN, Transformers, etc.

Model Selection and Training



Model Selection and Training



Model Evaluation



- Use appropriate metrics:
 - Regression: RMSE, R-squared, MAE
 - Classification: Accuracy, Precision, Recall, F1 score, Log loss
- Compare model performance against benchmarks or baseline models
- For time series: evaluate on chronologically ordered test set

Performance Metrics

Regression Problems:

- Mean Squared Error and Root Mean Squared Error:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- Mean Absolute Error:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- R squared:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y}_i)^2}$$

Performance Metrics

Classification Problems:

- Confusion matrix:

		Predicted Class	
		Positive	Negative
Actual Class	Positive	TP	FN
	Negative	FP	TN

- Accuracy:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Performance Metrics



Classification Problems with imbalanced classes:

- Precision:

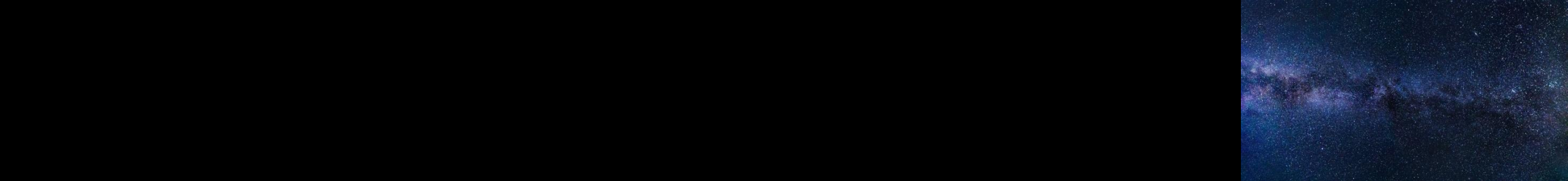
$$Precision = \frac{TP}{TP + FP}$$

- Recall:

$$Recall = \frac{TP}{TP + FN}$$

- F1 score:

$$F_1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$



Linear Models

Linear Regression



- Predicts a continuous output (y) based on input features (x)
- **Hypothesis:**

$$\hat{y}_i = \sum_{j=0}^M \beta_j x_{i,j}$$

- **Cost function:** Mean Squared Error (MSE)

$$L(\beta) = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Linear Regression



Key Components:

- Features (x): Input variables
- Parameters:
 - Coefficients (β): Weights for each feature
 - Intercept (β_0): Y-axis intercept

Objective: Minimize the difference between predicted ($\hat{y}$) and actual (y) values

Gradient Descent



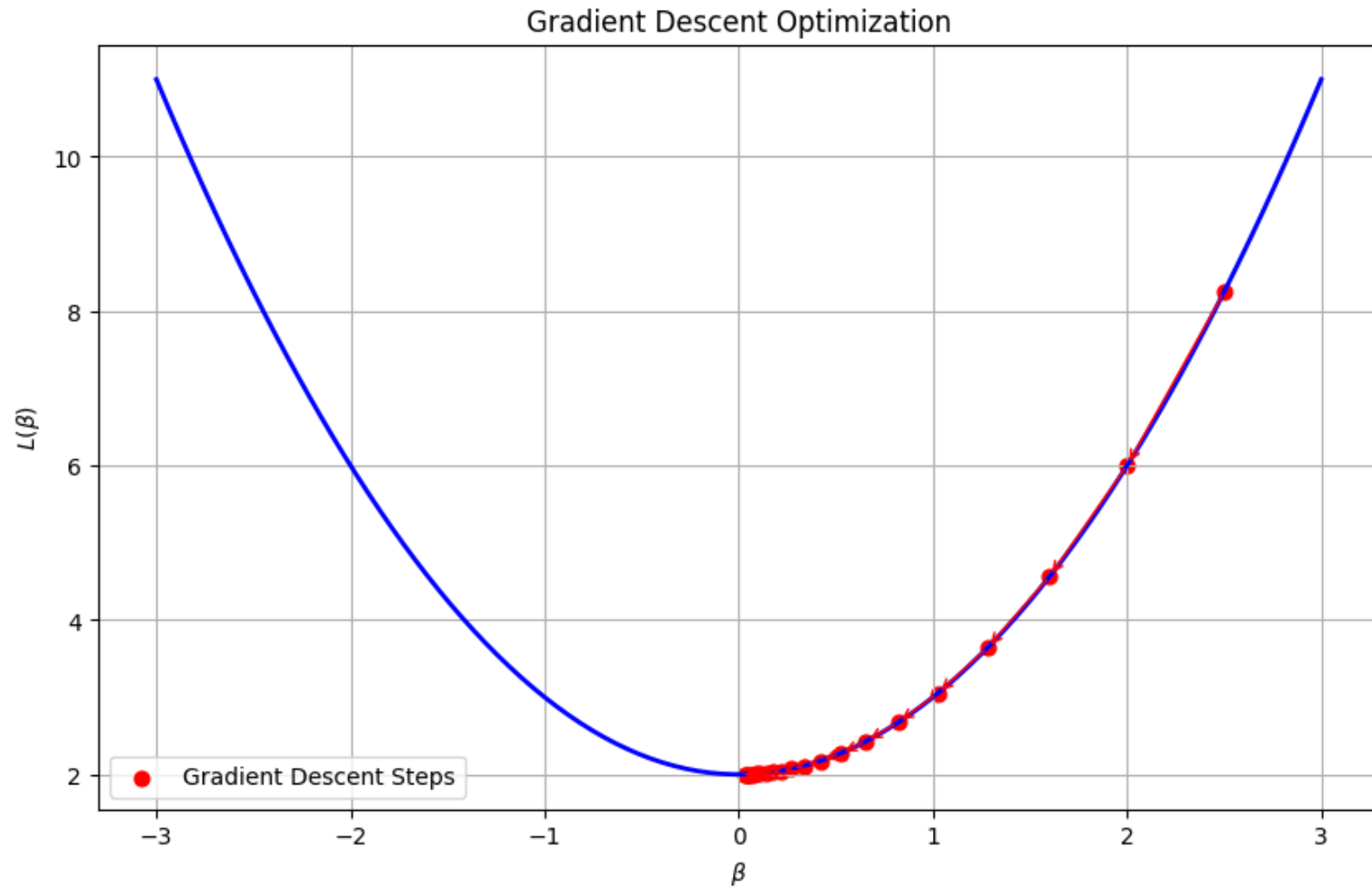
- Gradient descent (or steepest descent) is an iterative first-order optimisation algorithm used to find a local minimum/maximum of a given differentiable function
- This method is commonly used in machine learning to minimise a loss function (e.g. in a linear regression, neural networks, etc.)
- The algorithm iteratively calculates the next point using gradient at the current position, scales it (by a learning rate γ) and subtracts obtained value from the current position (makes a step).
- The process can be written as:

$$p_{n+1} = p_n - \gamma \nabla f(p_n)$$

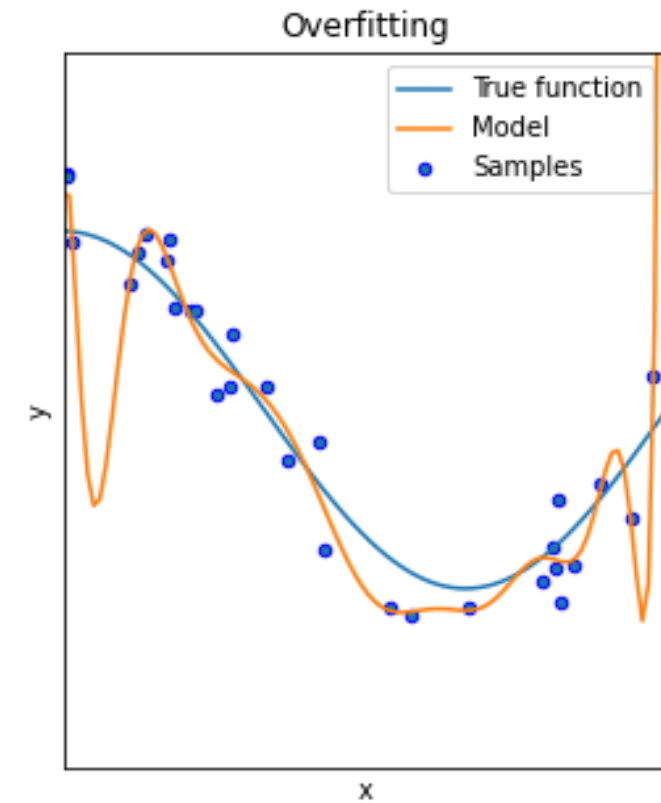
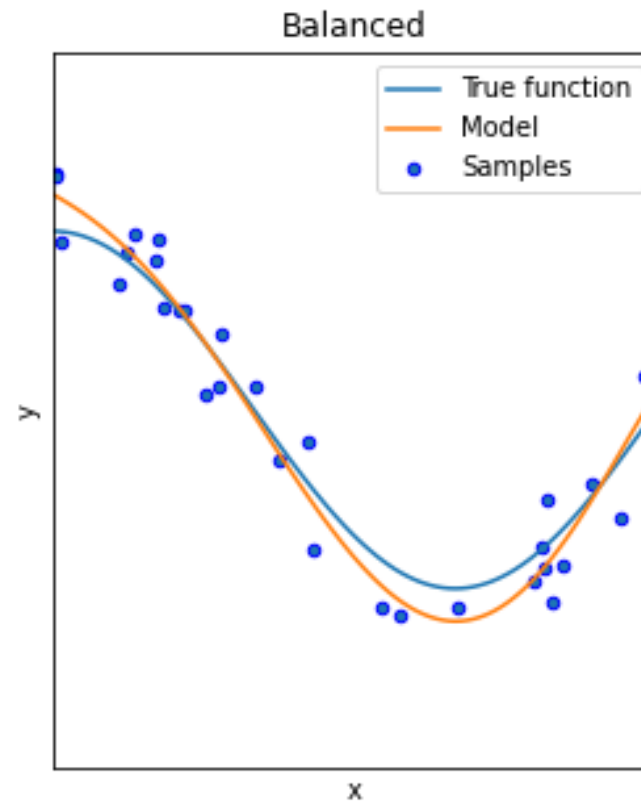
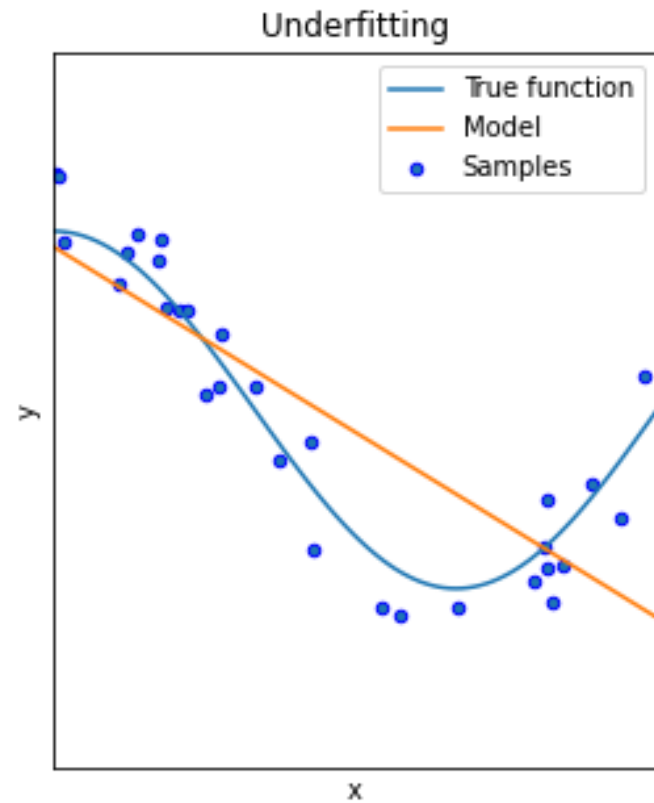
Where:

- p is a set of parameters that we need to estimate
- $\nabla f(p)$ is the gradient for n-dimensional
- γ is the learning rate

Gradient Descent



Linear Regression



Ridge Regression



- **Concept:**

- Extension of linear regression with regularization
- Adds a penalty term to the loss function

- **Hypothesis:**

$$\hat{y}_i = \sum_{j=0}^M \beta_j x_{i,j}$$

- **Cost function:** adding L2 regularization

$$L(\beta) = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=0}^M \beta_j^2$$

Ridge Regression



- The coefficients are shrunk towards zero, but not set to exactly zero

=> All features are kept, but their impact is reduced

Ridge produces dense coefficients: it shrinks, it does not select. Use Lasso when you want selection.

Why: the L2 penalty contributes a gradient $2 \cdot \lambda \cdot \beta$, which vanishes at $\beta = 0$, so nothing holds a coefficient exactly at zero. The L1 penalty of Lasso has a constant subgradient of $\pm \lambda$ at zero, and that is what creates exact zeros.

- Advantages:
 - Coefficient shrinkage
 - Reduce model complexity and overfitting

Ridge Regression

- Why β is not driven to exactly zero?

$$\hat{y}_i = \beta x_i$$

- Cost function:

$$L(\beta) = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \beta^2 = \frac{1}{2N} \sum_{i=1}^N (y_i - \beta x_i)^2 + \lambda \beta^2$$

Ridge Regression

- $N = 1$:

$$L(\beta) = (y - \beta x)^2 + \lambda \beta^2 = y^2 - 2\beta xy + x^2 \beta^2 + \lambda \beta^2$$

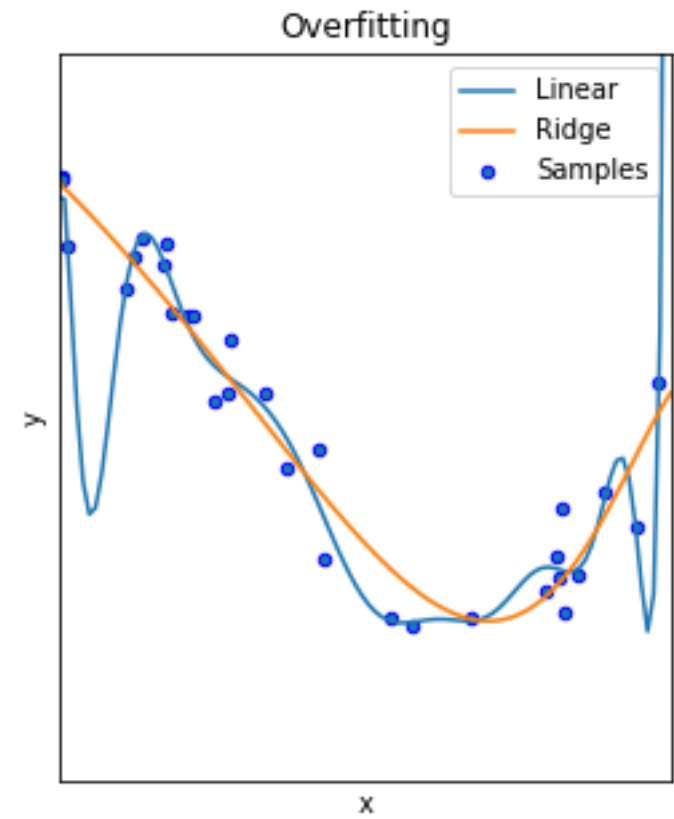
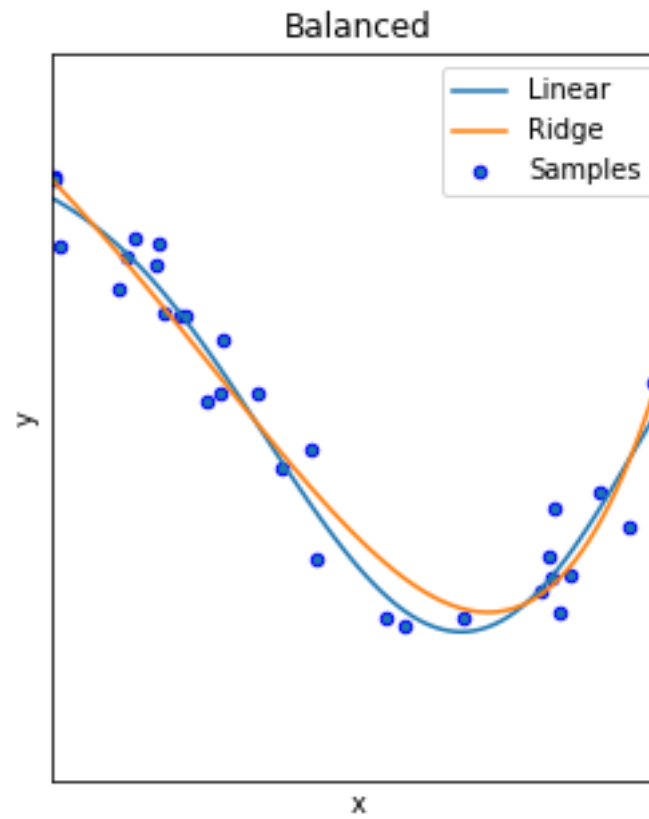
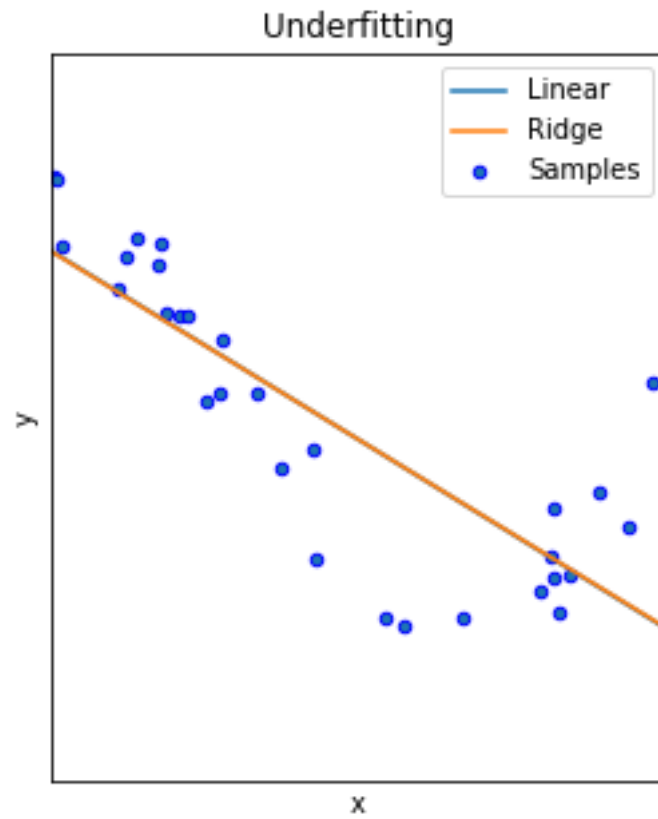
- To find $\text{Min } L(\beta)$ we take partial derivative w.r.t β :

$$-2xy + 2x^2\beta + 2\lambda\beta = 0$$

$$\Rightarrow x^2\beta + \lambda\beta = xy$$

$$\Rightarrow \beta = \frac{xy}{x^2 + \lambda} \quad \Rightarrow \beta \neq 0 \quad \forall \lambda$$

Ridge Regression



Lasso Regression



- **Concept:**

- Least Absolute Shrinkage and Selection Operator
- Another regularized version of linear regression

- **Hypothesis:**

$$\hat{y}_i = \sum_{j=0}^M \beta_j x_{i,j}$$

- **Cost function:** adding L1 regularization

$$L(\beta) = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=0}^M |\beta_j|$$

Lasso Regression



- Coefficients can go to zero => feature selection
- Advantages:
 - Performs both feature selection and regularization
 - Better for problem with many useless features
 - Reduce model complexity and overfitting

Lasso Regression

- Why β can go to zero?

$$\hat{y}_i = \beta x_i$$

- Cost function:

$$L(\beta) = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda |\beta| = \frac{1}{2N} \sum_{i=1}^N (y_i - \beta x_i)^2 + \lambda |\beta|$$

Lasso Regression

- $N = 1$:

$$L(\beta) = (y - \beta x)^2 + \lambda |\beta| = y^2 - 2\beta xy + x^2 \beta^2 + \lambda |\beta|$$

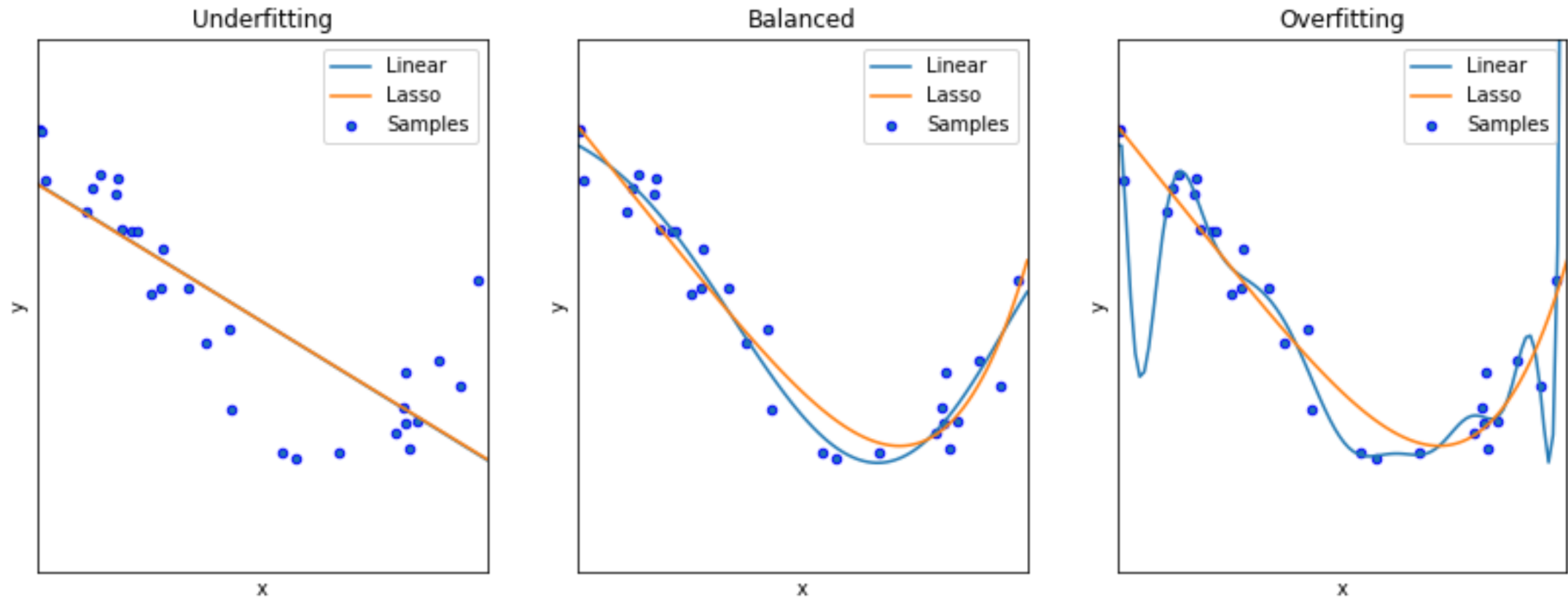
- To find $\text{Min } L(\beta)$ we take partial derivative w.r.t β in case $\beta > 0$:

$$-2xy + 2x^2\beta + \lambda = 0$$

$$\Rightarrow 2x^2\beta + \lambda = 2xy$$

$$\Rightarrow \beta = \frac{2xy - \lambda}{2x^2}$$

Lasso Regression



A Critical Prerequisite: Feature Scaling



The Problem: Unfair Penalties

A coefficient's size is determined by two things:

- Its **actual importance** (what we want to measure).
- Its **arbitrary original units** (which is just noise).

Consider a model with two features:

- Feature A: Annual_Salary (values from 50,000 to 1,000,000)
- Feature B: Years_Experience (values from 1 to 40)

To get the same impact on the output, the coefficient for Annual_Salary will be tiny (e.g., \$0.0003\$), while the coefficient for Years_Experience will be large (e.g., \$2,500\$).

A Critical Prerequisite: Feature Scaling



The Risk:

Ridge and Lasso will unfairly punish the Years_Experience feature (because its coefficient is large) and almost ignore the Annual_Salary feature. The model will be biased by its units, not its importance.

Scaling (Normalization or Standardization) converts all features to the same scale (e.g., mean of 0, standard deviation of 1).

Financial Analogy: This is like converting all your financial data—some in Euros, some in Yen, some in Dollars—into a single, common currency before you compare them.

Bottom Line: Scaling ensures the penalty is fair. The model can then find the true importance of each feature, regardless of its original units.

Ridge vs Lasso



Ridge Regression (The Stability Tool):

- Use When:
 - Most features are likely relevant
 - Dealing with multicollinearity
 - You want to keep all features in the model
- Why:
 - Ridge is a risk management tool that forces the model to be stable and robust. It's less sensitive to noise and produces reliable predictions.

Ridge vs Lasso



Lasso Regression (The Insight Tool):

- Use When:

- You have many features (e.g., 200 features for a credit application) and you don't know which one matter.
- You suspect only a few features are important
- You want to perform feature selection
- You need a simpler, more interpretable model

- Why:

- Lasso is a feature selection tool. By driving some coefficients to exactly zero, it gives you a simpler, more interpretable model. This is critical for explaining your model to a boss or a bank regulator.
- Caveat: among correlated features, Lasso keeps one almost arbitrarily and may drop a useful one.

Ridge vs Lasso



General Guidelines:

- Start with Lasso if you believe many features are irrelevant
- Use Ridge if you are dealing with multicollinearity or if you want to keep all features but reduce their impact
- Consider ElasticNet for a balance between the two approaches
- Cross-validation can help choose between Ridge and Lasso

Remember: The choice often depends on your domain knowledge and specific problem context!

ElasticNet



- **Hypothesis:**

$$\hat{y}_i = \sum_{j=0}^M \beta_j x_{i,j}$$

- **Cost function:** adding both L1 and L2 regularization

$$L(\beta) = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda_1 \sum_{j=0}^M |\beta_j| + \lambda_2 \sum_{j=0}^M \beta_j^2 = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda [\alpha \sum_{j=0}^M |\beta_j| + (1 - \alpha) \sum_{j=0}^M \beta_j^2]$$

- Best of both worlds

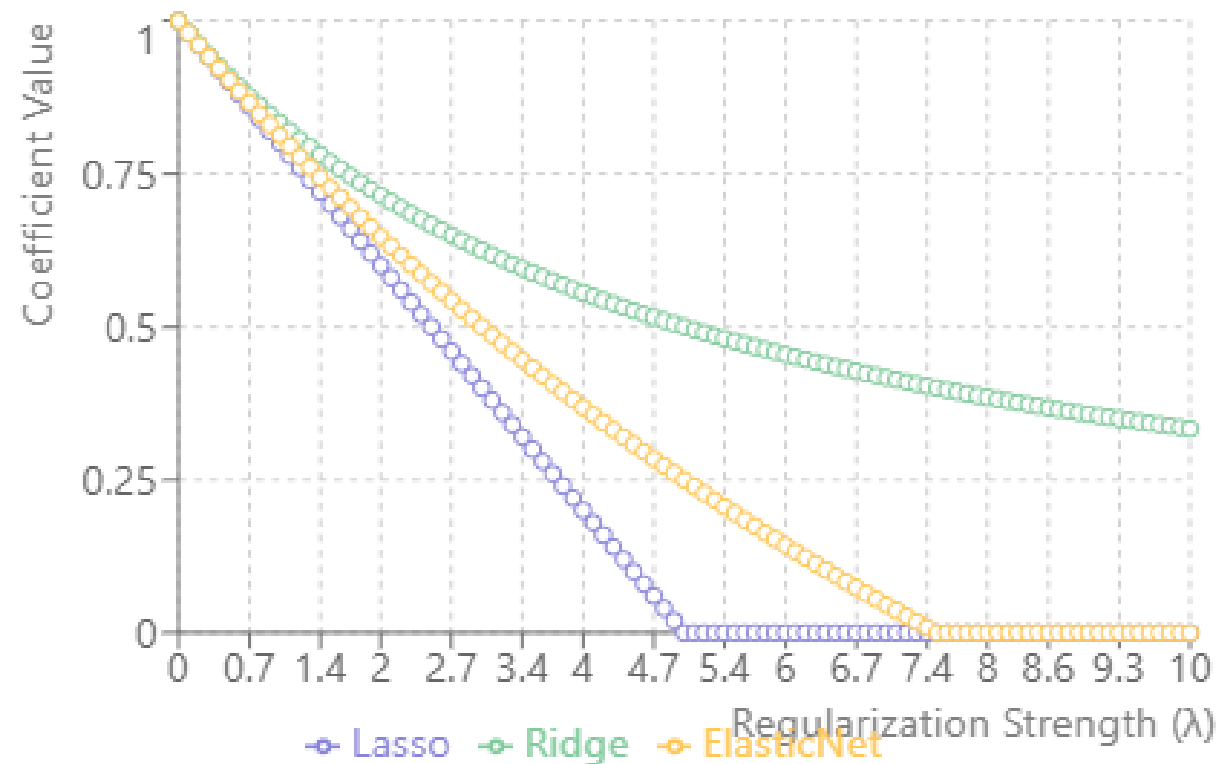
- Disadvantages:

- Computational cost: two penalty hyperparameters to tune (the overall strength and the L1/L2 mix), not twice as many coefficients

ElasticNet



Coefficient Paths: Lasso vs Ridge vs ElasticNet



Logistic Regression



- **Linear regression:**

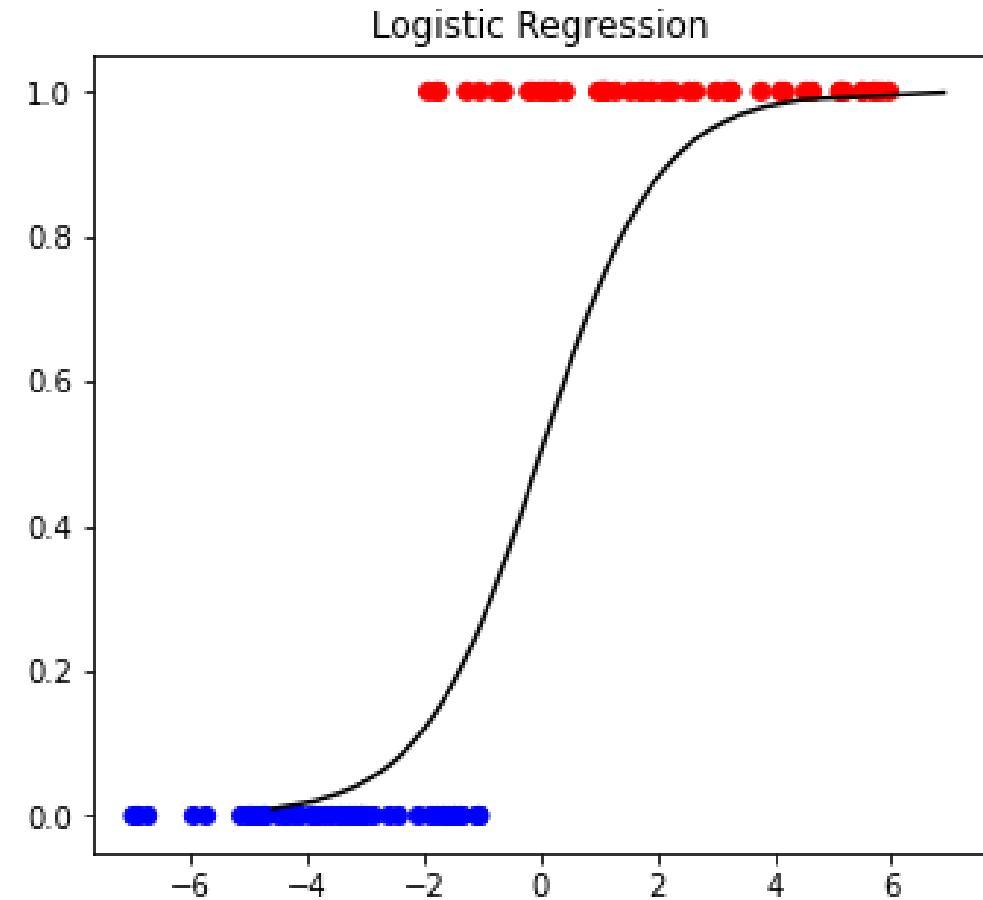
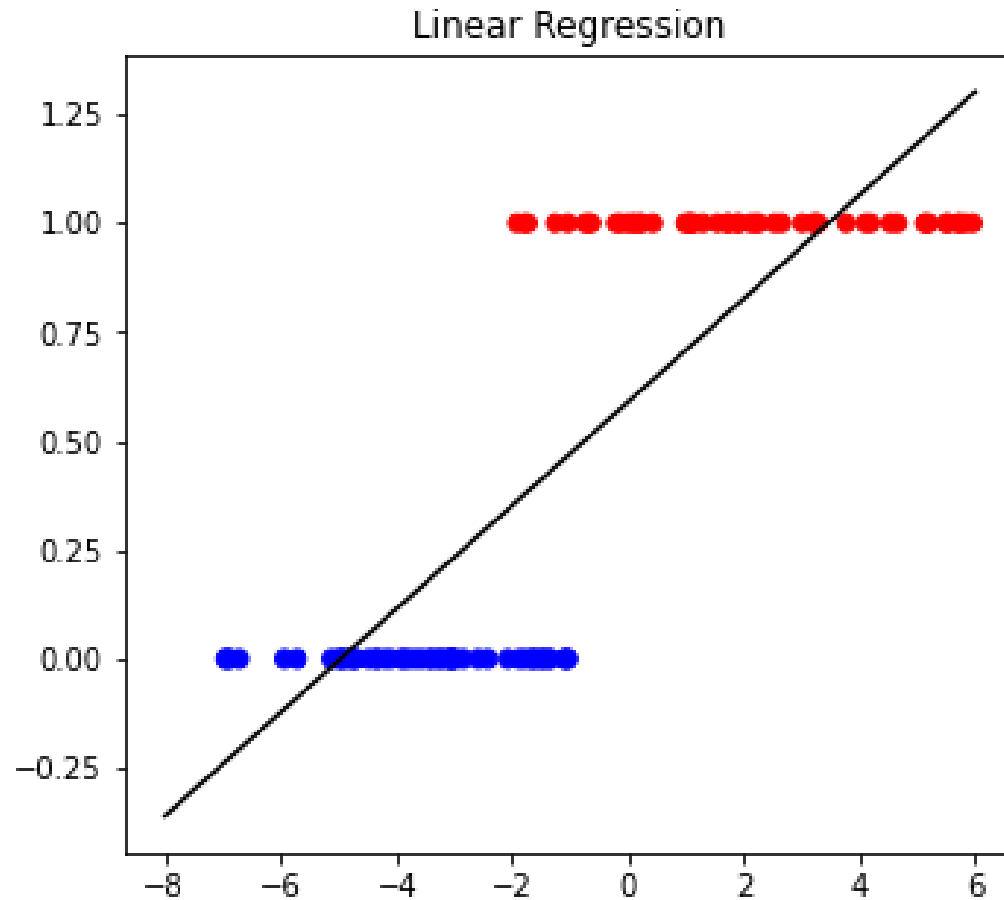
$$y_i \sim \sum_{j=0}^M \beta_j x_{i,j}$$

- This is not applicable for $y \in \{0,1\}$
- y_i can be realized as a random variables take the values one and zero with probabilities p_i and $1 - p_i$
- This p_i follow the Bernoulli distribution and we can formulate the problem as:

$$p_i \sim \sum_{j=0}^M \beta_j x_{i,j}$$

- Again since $p \in [0,1]$, the linear regression is also not useful

Logistic Regression



Logistic Regression



- **Logistic equation:**

$$p_i = \frac{e^{\sum_{j=0}^M \beta_j x_{i,j}}}{1 + e^{\sum_{j=0}^M \beta_j x_{i,j}}}$$

- This equation can be linearized by the following transformation

$$\text{logit}(p_i) = \ln\left(\frac{p_i}{1 - p_i}\right) = \sum_{j=0}^M \beta_j x_{i,j}$$

Logistic Regression



- **The maximum log-likelihood function:**

$$\ln(L) = \frac{1}{N} \sum_{i=1}^N [(1 - y_i) \ln(1 - p_i) + y_i \ln(p_i)] = \frac{1}{N} \sum_{i=1}^N \left[\ln(1 - p_i) + y_i \ln\left(\frac{p_i}{1 - p_i}\right) \right]$$

- Log loss or logistic loss function by substitute p_i into the function:

$$L_{log} = -\ln(L) = -\frac{1}{N} \sum_{i=1}^N \left[-\ln\left(1 + e^{\sum_{j=0}^M \beta_j x_{i,j}}\right) + y_i \left(\sum_{j=0}^M \beta_j x_{i,j}\right) \right]$$

Logistic Regression



- The gradient of L_{log} :

$$\begin{aligned}\frac{\partial L}{\partial \beta_j} &= -\frac{1}{N} \sum_{i=1}^N \left[-\frac{x_{i,j} e^{\sum_{j=0}^M \beta_j x_{i,j}}}{1 + e^{\sum_{j=0}^M \beta_j x_{i,j}}} + y_i x_{i,j} \right] = -\frac{1}{N} \sum_{i=1}^N \left(y_i - \frac{e^{\sum_{j=0}^M \beta_j x_{i,j}}}{1 + e^{\sum_{j=0}^M \beta_j x_{i,j}}} \right) x_{i,j} \\ &= \frac{1}{N} \sum_{i=1}^N (p_i - y_i) x_{i,j}\end{aligned}$$

Logistic Regression



- **Regularization for Logistic regression:**
- **Log loss function:**
 - Adding L1 penalty

$$L_{log} + \lambda \sum_{j=0}^M |\beta_j|$$

- Adding L2 penalty

$$L_{log} + \lambda \sum_{j=0}^M \beta_j^2$$

Logistic Regression



- **Log loss function:**

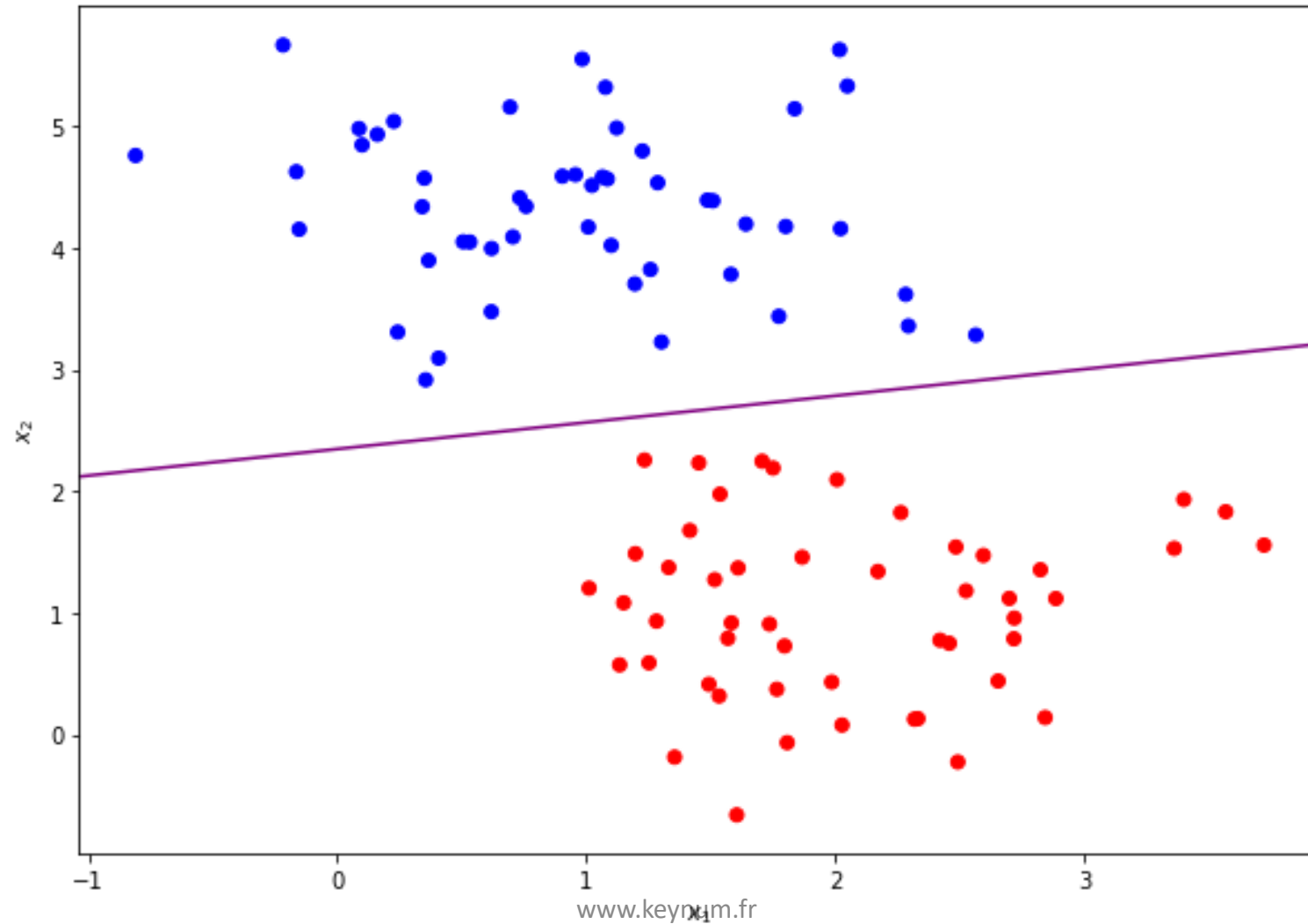
- Adding both L1 and L2 penalty

$$L_{log} + \lambda_1 \sum_{j=0}^M |\beta_j| + \lambda_2 \sum_{j=0}^M \beta_j^2$$

- or

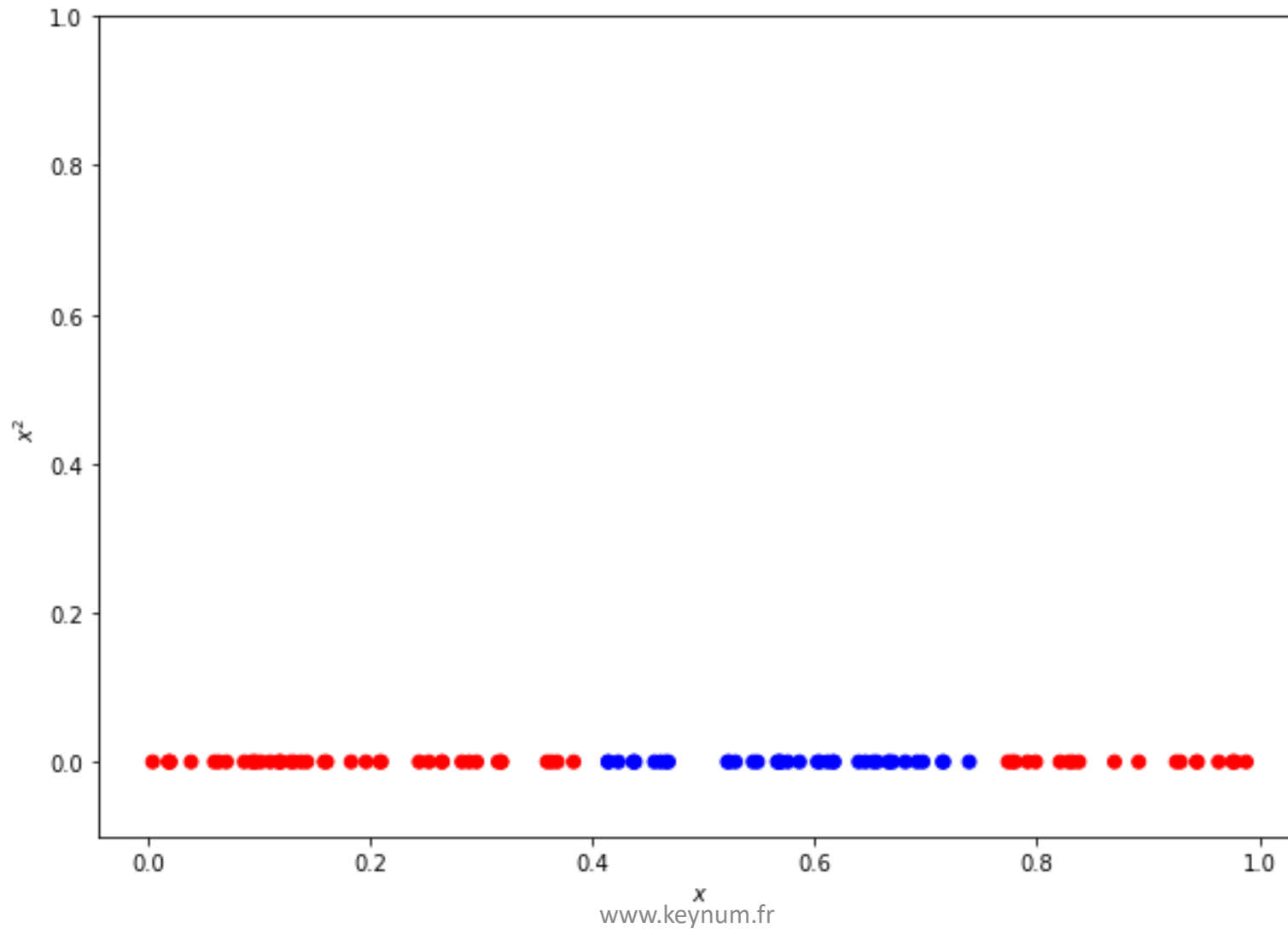
$$L_{log} + \lambda \sum_{j=0}^M (\alpha \beta_j^2 + (1 - \alpha) |\beta_j|)$$

Logistic Regression

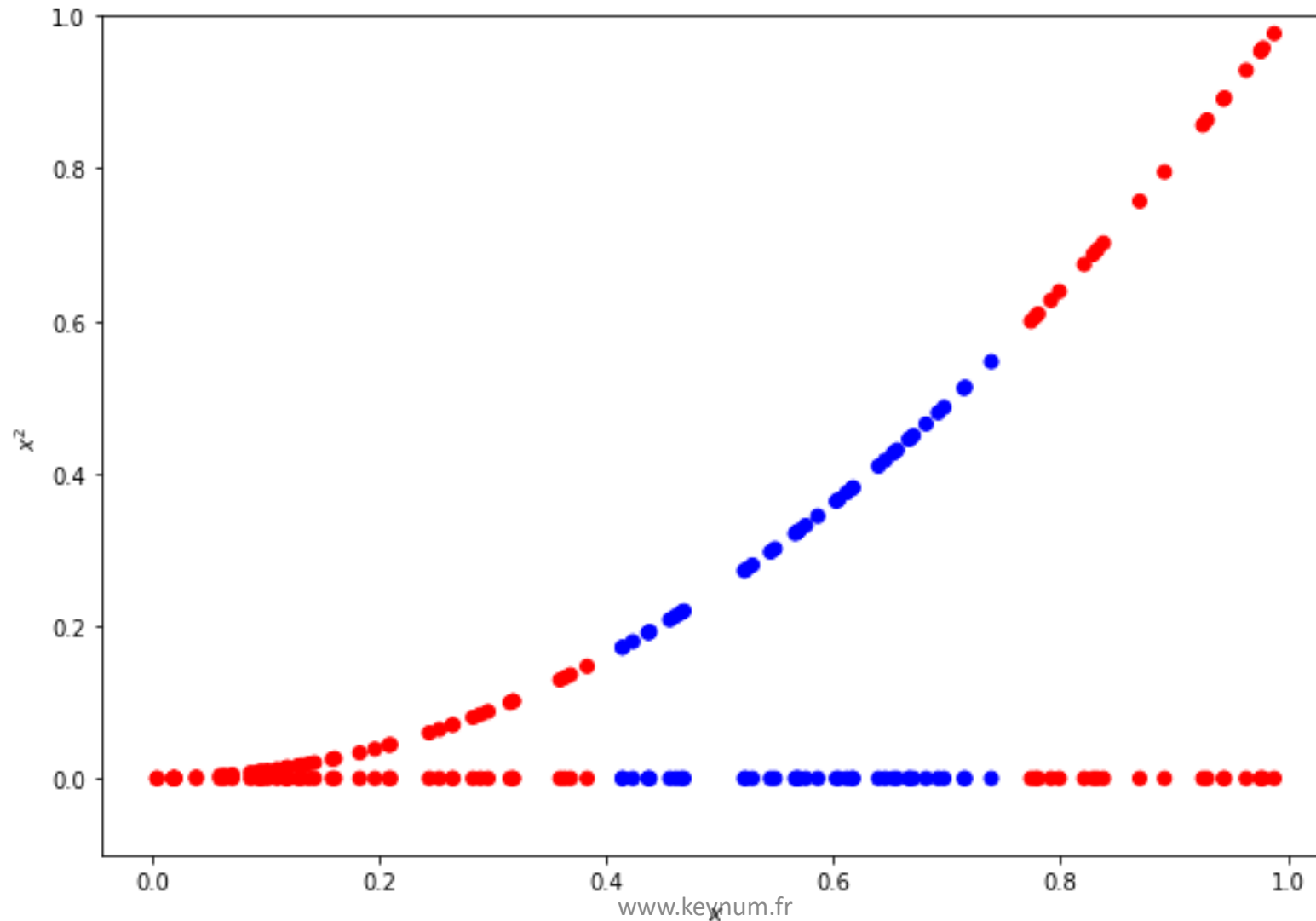




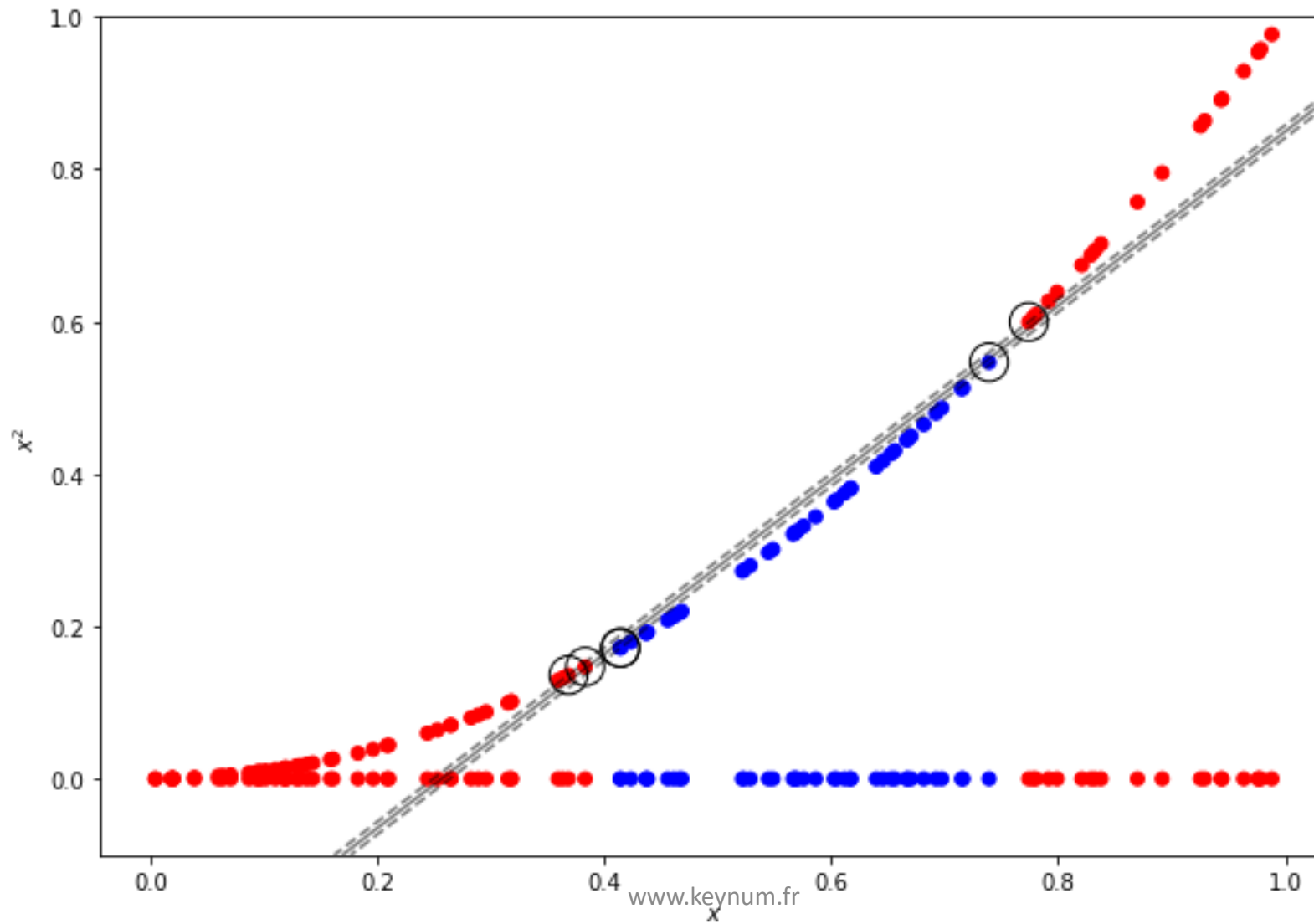
Logistic Regression



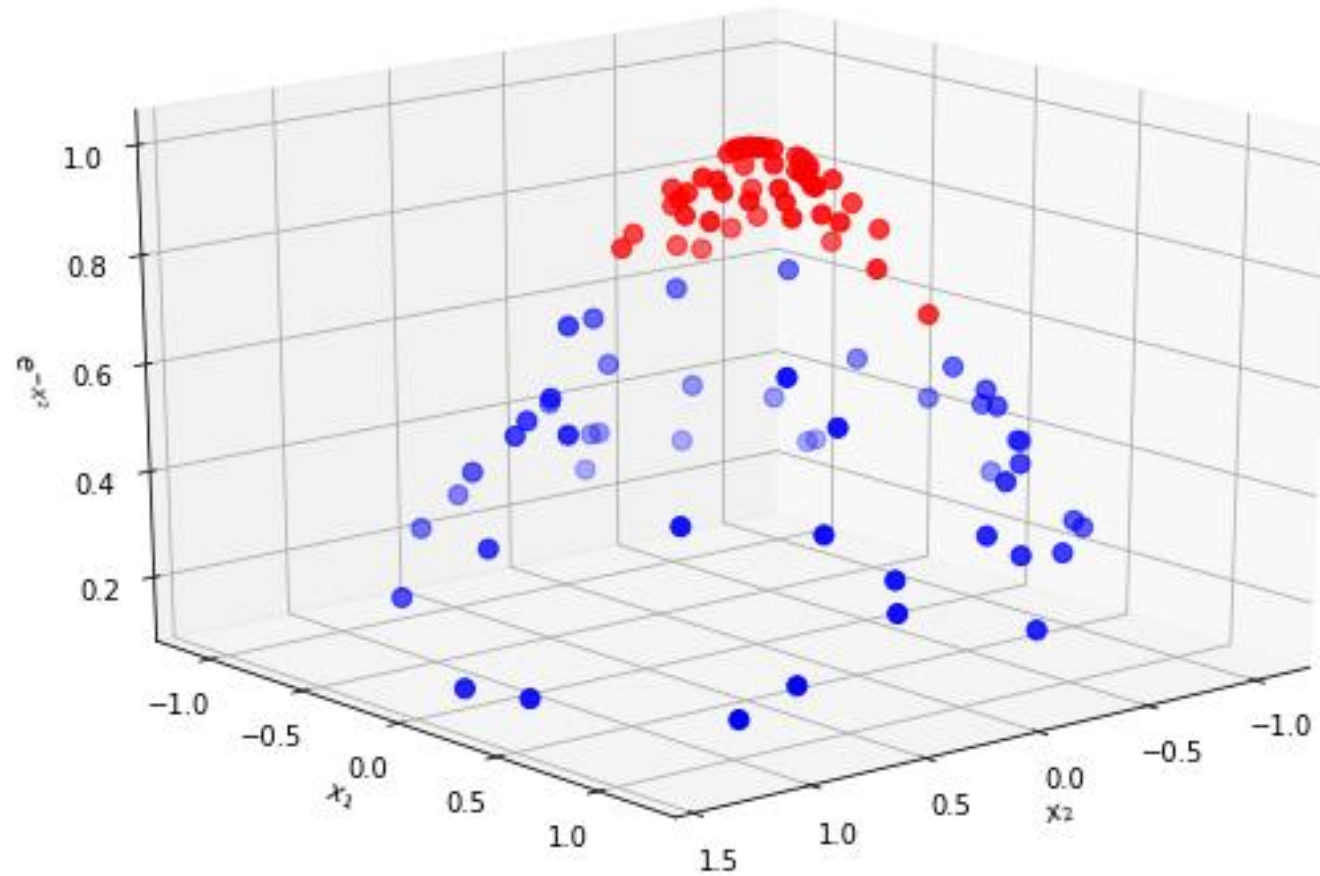
Logistic Regression

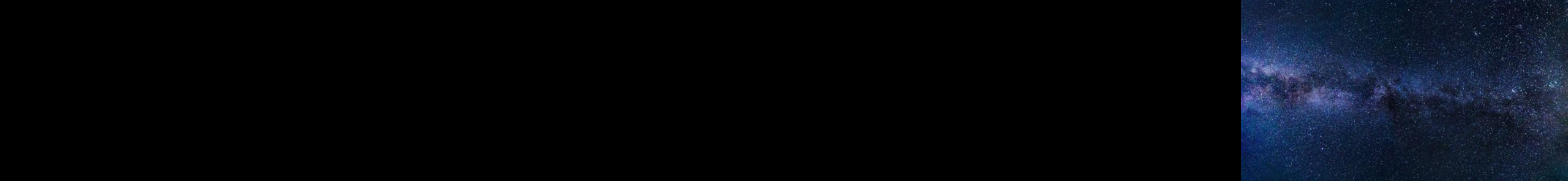


Logistic Regression



Logistic Regression





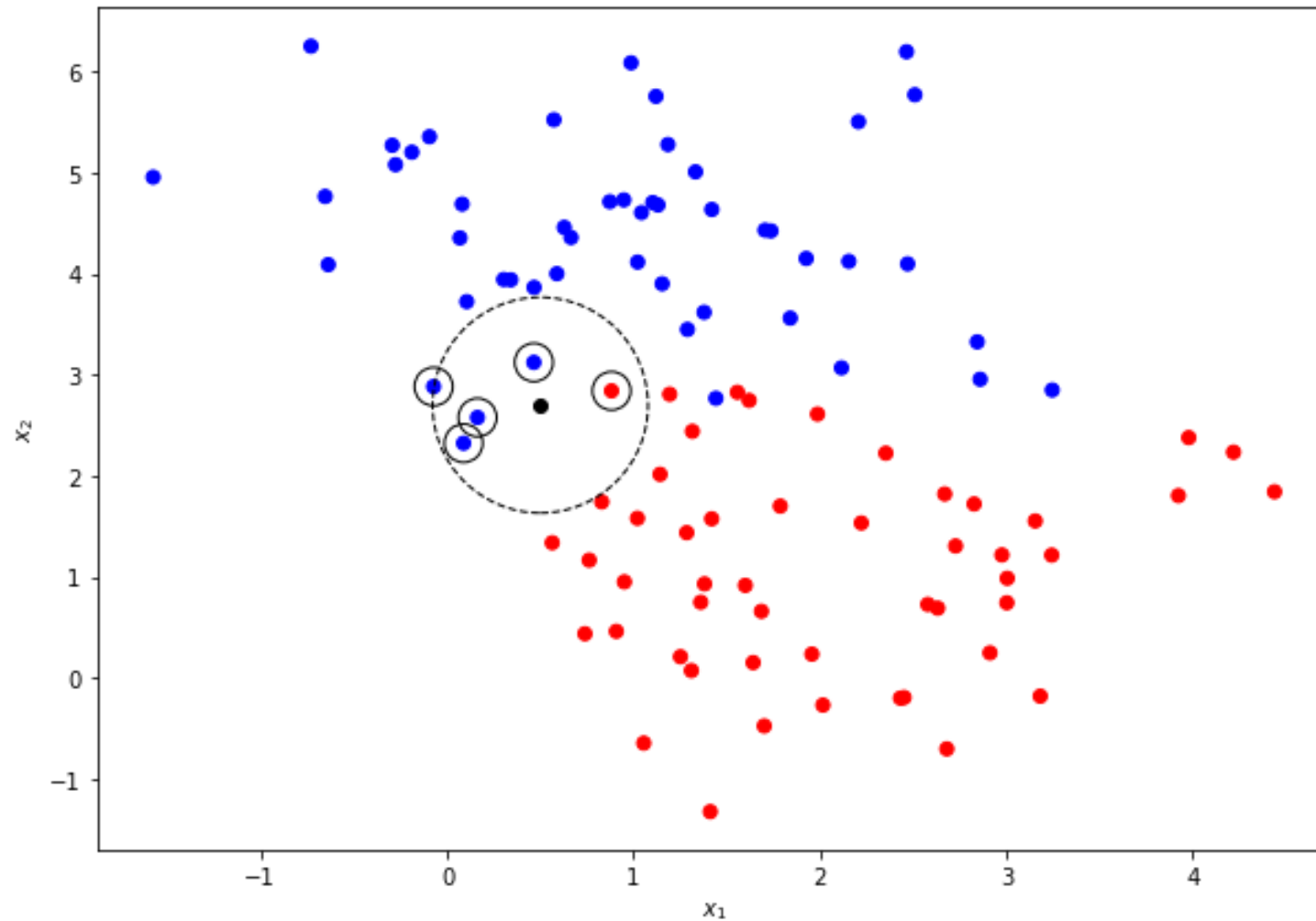
K-Nearest Neighbours

K-Nearest Neighbours



- K-nearest neighbor (KNN) is a very simple algorithm in which each observation is predicted based on its “similarity” to other observations.
- KNN is a memory-based algorithm and cannot be summarized by a closed-form model.
- The KNN algorithm identifies k observations nearest to the new record being predicted and then uses:
 - The average of those k observations as the predicted output for regression
 - The most common class of those k observations as the predicted output for classification.
- Features must be rescaled for this type of models

K-Nearest Neighbours



K-Nearest Neighbours



Distance measure:

- Euclidean distance:

$$\sqrt{\sum_{j=1}^M (x_{aj} - x_{bj})^2}$$

- Manhattan distance:

$$\sum_{j=1}^M |x_{aj} - x_{bj}|$$

K-Nearest Neighbours



Distance measure:

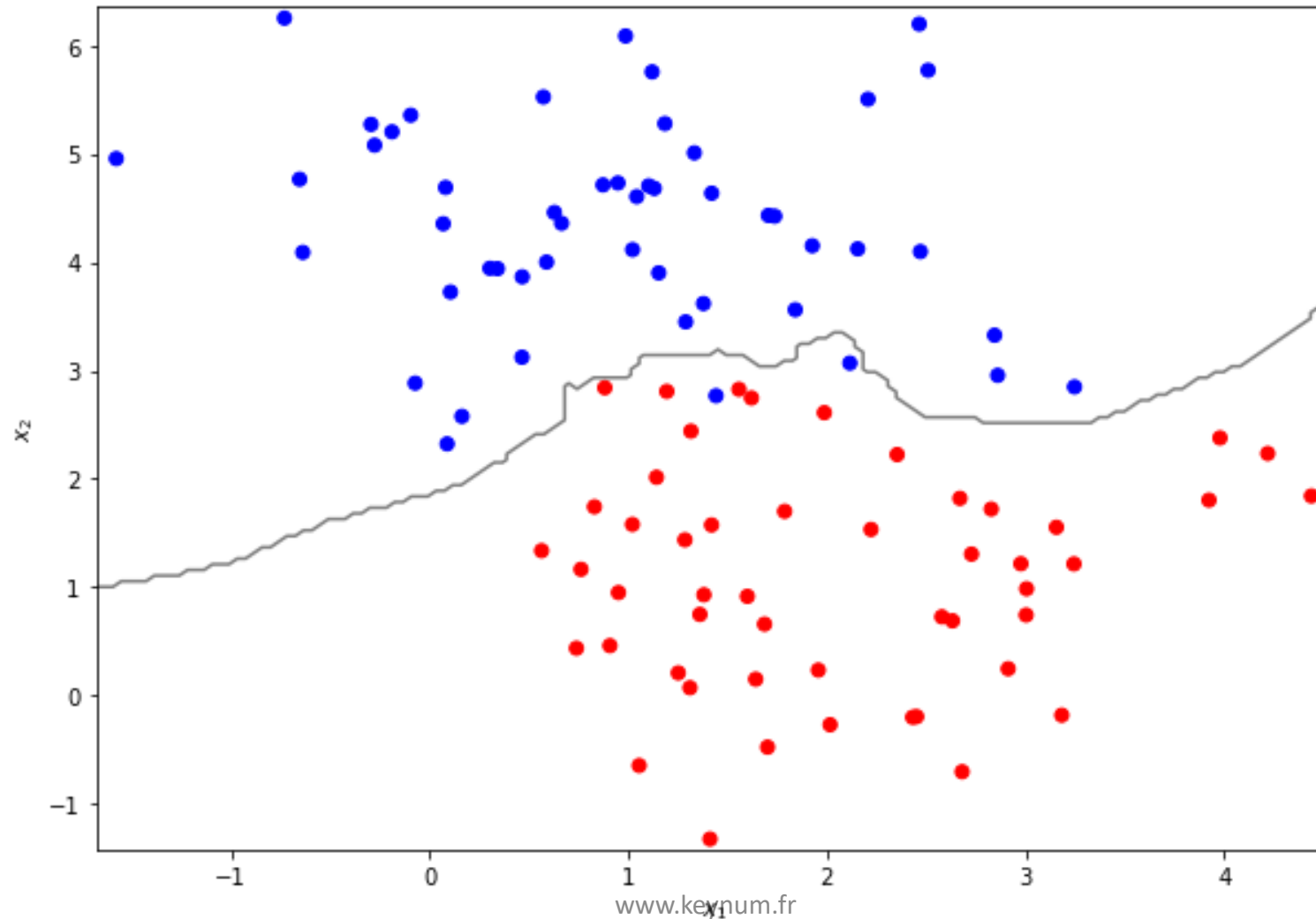
- Minkowski distance: a generalization of the Euclidean and Manhattan distances

$$\left(\sum_{j=1}^M |x_{aj} - x_{bj}|^q \right)^{\frac{1}{q}}$$

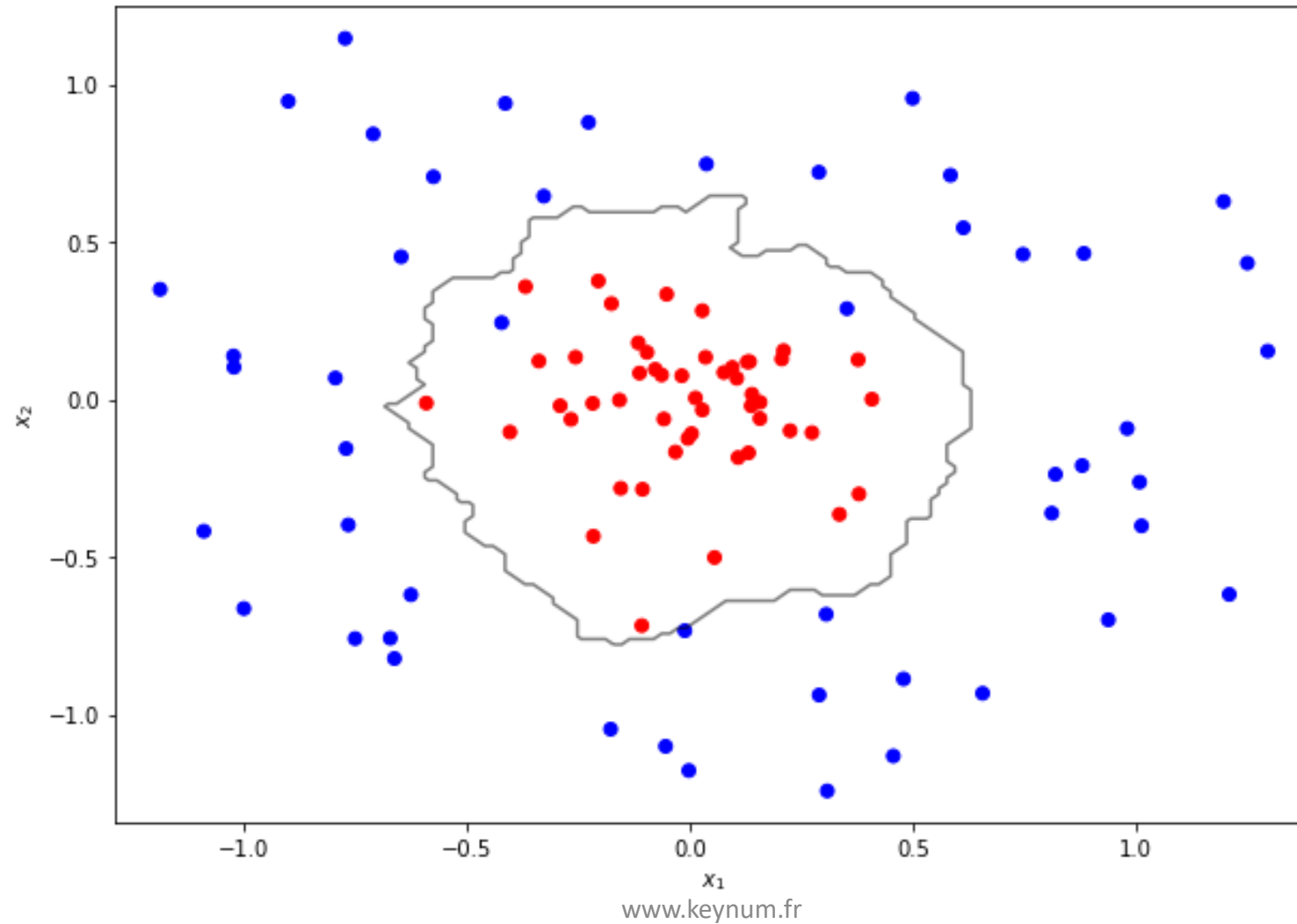
where $q > 0$

- When $q = 2$: Euclidean distance
- When $q = 1$: Manhattan distance

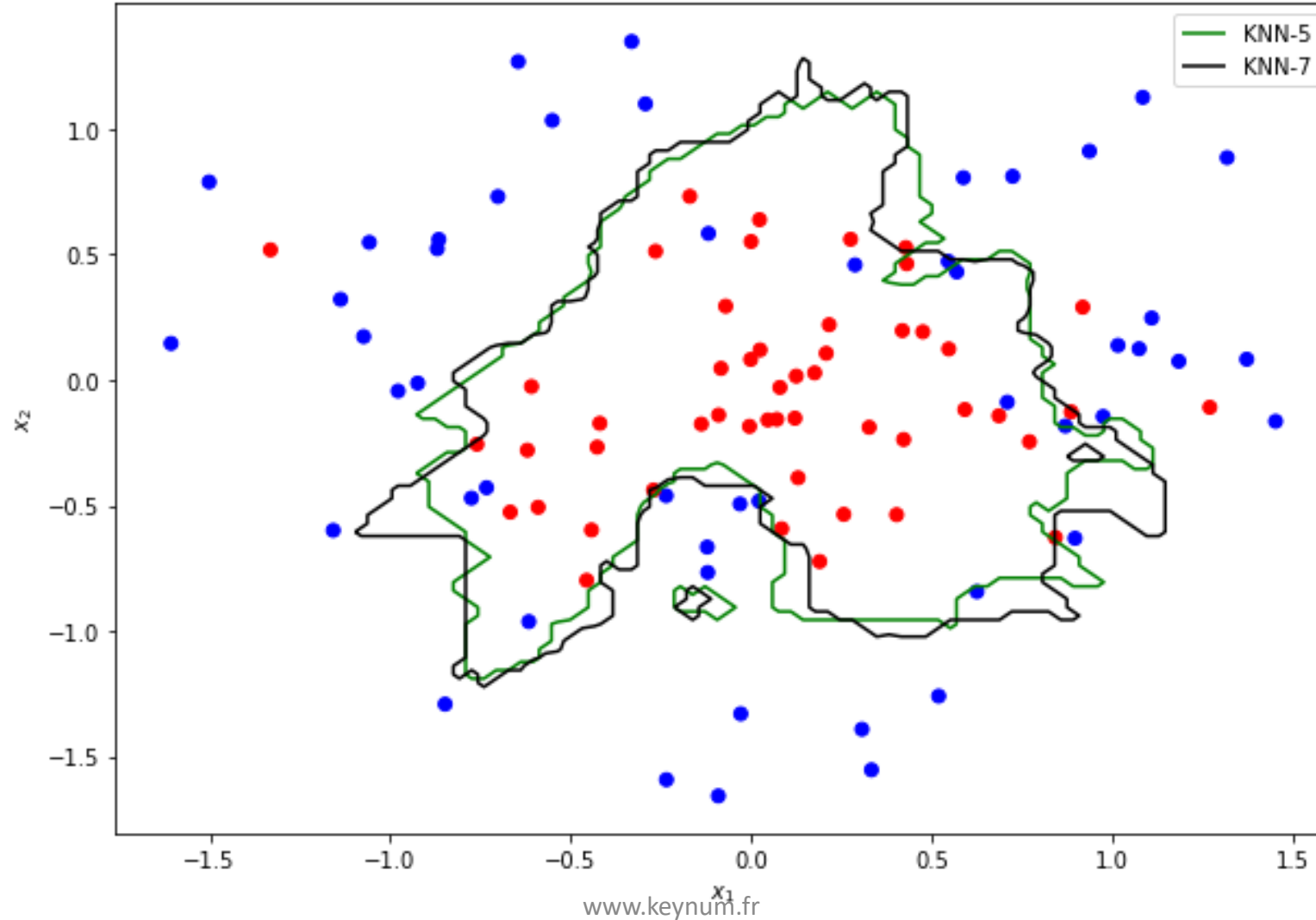
K-Nearest Neighbours



K-Nearest Neighbours



K-Nearest Neighbours



K-Nearest Neighbours



- The performance of KNNs is very sensitive to the choice of k , low values of k typically overfit and large values often underfit.
- When using KNN for classification, it is best to assess odd numbers for k to avoid ties in the event there is equal proportion of response levels (i.e. when $k = 2$ one of the neighbours could have class 0 while the other neighbour has class 1)

From Scores to Decisions: The Threshold

- A classifier outputs a score or a probability. `predict()` cuts it at 0.5.
- 0.5 is a convention, not an optimum, and rarely the right operating point in finance.
- Lower threshold: catch more positives, raise more false alarms.
- Higher threshold: fewer false alarms, miss more real cases.
- The model is fixed. The threshold is a business decision, chosen on the validation set.
- ROC curve: true positive rate against false positive rate. AUC summarises ranking only.
- Precision-Recall curve: more informative when positives are rare (fraud, default).

Choosing the Threshold from Costs

- Give each error a cost, then minimise expected cost instead of maximising accuracy:
$$\text{expected cost} = C_{\text{FN}} \times \text{FN} + C_{\text{FP}} \times \text{FP}$$
- Credit: a default costs the loss given default; a rejected good client costs the margin.
- Fraud: a missed fraud costs the transaction; a false alarm costs a blocked card.
- Sweep the threshold on validation, take the minimum, then report on the test set.
- Imbalance: `class_weight` helps the model fit, the threshold sets the operating point.

Probability Calibration



- Calibrated: among the cases scored at 20%, about 20% actually experience the event.
- Ranking and calibration differ. AUC 0.95 is compatible with badly wrong probabilities.
- Expected loss = $PD \times LGD \times EAD$. An uncalibrated PD misprices the loan and the capital.
- A cost-based threshold also assumes the probability means what it says.
- What to expect in practice:
 - logistic regression is usually well calibrated, since it optimises log loss directly
 - SVM and boosted trees are over-confident; random forests are under-confident

Checking and Fixing Calibration



- Reliability diagram: bin the predictions, plot observed frequency against predicted.
 - a perfectly calibrated model lies on the diagonal
- Brier score: mean squared error of the predicted probability. Lower is better.
- CalibratedClassifierCV maps raw scores to calibrated probabilities:
 - "sigmoid" (Platt scaling): parametric, works with little data
 - "isotonic": more flexible, but needs more data or it overfits
- Calibrate on data the model has not seen. Fit, calibrate, then choose the threshold.